DESIGN AND IMPLEMENTATION OF AN AUTOMATED COURSE ALLOCATION SYSTEM FOR LECTURERS

BY

ABAYOMI, PROSPER GBOLAHAN

(2020/9526)

A FINAL YEAR PROJECT SUBMITTED TO THE DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY, COLLEGE OF NATURAL AND APPLIED SCIENCES

IN PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR THE AWARD OF BACHELOR OF TECHNOLOGY (B.TECH) DEGREE IN COMPUTER SCIENCE, BELLS UNIVERSITY OF TECHNOLOGY, OTA, OGUN STATE, NIGERIA

JULY, 2025

DECLARATION

I ABAYOMI PROSPER hereby declare that the project **DESIGN AND IMPLEMENTATION OF AN AUTOMATED COURSE ALLOCATION SYSTEM FOR LECTURERS** was carried out by me, under the supervision of DOCTOR MRS. E.A. OLUWATOSIN of the Department of Computer Science and Information Technology, College of Natural and Apllied Sciences, Bells University of Technology, Ota, Nigeria. I attest that the project has not been presented either wholly or partially for the award of any degree elsewhere. All sources of data and scholarly information used in this project are duly acknowledged using references.

..…….. ..…….

ABAYOMI PROSPER **Date**

CERTIFICATION

I certify that this project titled "**DESIGN AND IMPLEMENTATION OF AN AUTOMATED COURSE ALLOCATION SYSTEM FOR LECTURERS**" was carried out by ABAYOMI PROSPER (2020/9526) under my supervision in the Department of Computer Science and Information Technology, College of Natural and Applied Sciences, Bells University of Technology, Ota, Ogun State, Nigeria.

…… ……

DR. MRS. A.E. OLUWATOSIN **DATE**

(Project Supervisor)

…… ……

DR. J.O.J EZIKE **DATE**

(Head Of Department)

DEDICATION

This work is dedicated to God Almighty, my mother, my siblings, my closest friends, my beloved coursemates, and all who have been by my side. Your unwavering support, love, and encouragement have been the driving force behind this endeavor. Through the highs and lows of this journey, your belief in me has been a constant source of inspiration. This accomplishment is as much yours as it is mine. Thank you for being my pillars of strength and for sharing in the joy of reaching this milestone.

With love and gratitude,

Abayomi Prosper.

ACKNOWLEDGMENTS

I want to express my sincere gratitude to all those who have contributed to the completion of this thesis. First and foremost, I am deeply thankful to the Head of Department, Dr. J.O.J. Ezike, for his guidance, support, and invaluable insights throughout the research process and towards the completion of the project. His mentorship and words of advice have been instrumental in shaping the direction of this work. Heartfelt thanks to my project supervisor, Dr. A.E. Oluwatosin, for her constant support and mentorship throughout the project I extend my appreciation to the faculty and staff of the Bells University of Technology, whose academic expertise and resources have enriched my learning experience. Special thanks to Prof. Iwasoku, Dr. M.A. Adegoke, Dr. J.A Adeyiga, Dr. Ogunbiyi, Dr. Michael, Dr. O.A. Balogun, Dr. Ubaka, Mr. Ashade Abidemi, Mr. Temitope, Mr. Moses Achaz, Mr. Olaleye, and Mrs. Ibidapo, for their constant assistance and encouragement.

I am grateful to my mother, my brother, and my sister, who provided me with encouragement and constructive feedback. Your collaborative spirit has made this journey more enjoyable and rewarding. Heartfelt thanks to my coursemates and friends for their unwavering support, understanding, and encouragement. Your belief in me has been a constant source of motivation.

Finally, I want to express my gratitude to all the participants and sources who contributed to the project design and implementation process. Without their cooperation, this research would not have been possible. Thank you to everyone who played a part, big or small, in the realization of this project. Your support has been truly invaluable.

ABSTRACT

Universities and Higher Education Institutions (HEIs) frequently face challenges in the manual course allocation process, resulting in unequal workloads, administrative delays and subjective decisions. This project presents an Automated Course Allocation System (CAS) for Bells University of Technology that resolves such problems by optimally assigning instructors to courses whilst keeping in line with institutional constraints.

The Automated CAS uses a Linear Programming model to align lecturer expertise and preference (encoded as a ”Suitability score”) with workload constraints (1–15 units per week) and regulations (e.g, at most three instructors per course). The system, developed using a hybrid Agile–Waterfall process, is a web application, with a React/Tailwind CSS frontend and Node.js/SQL backend. Experiments on synthetic data (30 courses, 10 lecturers) achieved 94 % assignment accuracy, 2.7 s mean solve time, 92.1 % satisfaction (trade-off between fairness, expertise fit and constraint satisfaction) and 90 % manual effort reduction.

With the application of equitable, rules-based allocations, the CAS improves efficiency, fairness and stakeholder satisfaction. Its current limitations are that it is department-specific to Computer Science and Information Teachnology and does not have any real-time adjustment capabilities. Future improvements ought to involve AI-driven demand forecasting, extension to multiple departments and dynamic reallocation within the semester.

TABLE OF CONTENTS

DECLARATION ..... **I**

CERTIFICATION ..... **II**

DEDICATION ..... **III**

ACKNOWLEDGMENTS ..... **IV**

ABSTRACT ..... **V**

TABLE OF CONTENTS ..... **VI**

LIST OF FIGURES ..... **VII**

LIST OF TABLES ..... **VIII**

ABBREVIATIONS AND MEANINGS ..... **IX**

CHAPTER ONE **1**

INTRODUCTION **1**

1.1 Background of The Study 1

1.2 Problem Statement 3

1.3 Aim and Objectives of the Study 5

1.3.1 Aim of the study 5

1.3.2 Objectives of the study 5

1.4 Scope and Limitations of the Study 5

1.4.1 Scope of the study 5

1.4.2 Limitation of the study 5

1.5 Significance of the Study 7

CHAPTER 2 ..... **9**

LITERATURE REVIEW **9**

2.1 Introduction 9

Web Applications in HEIs 9

Uses of Web Applications in HEIs 10

2.2 Key Concepts in Course Allocation 11

2.2.1 Constraints....11
2.3 Brief History of Course Allocation in HEIs....12
2.4 Traditional Approaches to Course Allocation....14
2.4.1 Manual approaches....14
2.4.2 Random allocation....15
2.5 Optimization Algorithms for Course Allocation....15
2.5.1 Example linear programming approach to course allocation....17
2.6 Related works....20
2.6.1 Mathematical Representation....21
2.6.2 Relation to Proposed Project....23
2.6.3 Contrast From Existing Work....24
2.7 Advanced Approaches for Course Allocation System....28
2.7.1 Multi-objective optimization....29
2.7.2 Heuristic and meta-heuristic techniques....30
2.8 Typical Operation of a CAS....31
2.9 Evaluation of CAS in HEIs....34
2.9.1 Simulation and scenario testing....34
2.10 Existing Implementations/Case Studies of CAS in HEIs....35
2.11 Other Applications of Allocation Algorithms....37
2.11.1 Student admission allocation....37
2.11.2 Faculty and staff allocation....38
2.11.3 Classroom and Facility Allocation....39
2.11.4 Financial resource allocation....40
CHAPTER THREE....43
METHODOLGY....43
3.1 Introduction....43
3.2 System Development Process....43
3.2.1 Development methodology....44

3.2.2 Development phases..... 45
3.3 Tools and Technologies..... 47
3.3.1 Frontend development.....47
3.3.2 Backend development..... 48
3.3.3 Algorithm and deployment tools.....48
3.3.4 Development tools..... 49
3.4 System Workflow..... 49
3.5 System Architecture..... 53
3.5.1 Presentation layer..... 53
3.5.2 Business logic layer..... 53
3.5.3 Data layer (Database)..... 54
3.5.4 Decision points.....56
3.6 Mathematical Model of LP Algorithm..... 56
3.6.1 Factors considered.....53
3.6.2 Mathematical representation..... 56
3.7 Evaluation and Testing..... 61
3.7.1 Evaluation metrics.....62
3.7.2 Test Procedure..... 62
CHAPTER FOUR..... 64
RESULTS AND DISCUSSIONS..... 64
4.1 Introduction..... 64
4.2 Implementation Results..... 64
4.2.1 Frontend Results..... 64
4.2.2 Backend Results..... 75
4.2.3 Mathematical Model Results..... 78
4.3 Comparison with Previous Studies/Works..... 83
4.4 Performance Results..... 83
4.5 Limitations of the Study..... 85
CHAPTER FIVE..... 87

CONCLUSIONS AND RECOMMENDATIONS..... 87

5.1 Conclusion.....87

5.2 Recommendations for Future Research..... 88

REFERENCES.....90

APPENDICES..... 95

LIST OF FIGURES

Fig 2.1 Workflow structure of a typical Course Allocation System……... 31

Fig 3.1 Agile-Waterfall Hybrid Approach……..……... 46

Fig 3.2 ACAS Workflow Flowchart……...……... 51

Fig 3.3 ACAS Sequence Diagram...……... 52

Fig 3.4 ACAS SAD (System Architecture Diagram)……... 55

Fig 3.5 ACAS Use Case Diagram……... 57

Fig 4.1 Landing Page……...……... 67

Fig 4.2 Admin Login Page……...……... 68

Fig 4.3 Lecturer Login Page……,.. 69

Fig 4.4 Lecturer Dashboard……... 70

Fig 4.5 Admin Dashboard...……... 71

Fig 4.6 Lecturer Details Table……..……... 72

Fig 4.7 Admin Allocation Table……..……... 73

Fig 4.8 Form to Add Courses……...……... 74

Fig 4.9 Modal for Course Details……... 77

Fig 4.10 Allocation Algorithm……...……. 80

Fig 4.11 Algorithm Implementation…….……. 81

Fig 4.12 Algorithm Implementation 2……..……. 82

LIST OF TABLES

Table 2.1 Related works……..…25

ABBREVIATIONS AND MEANINGS

Abbrevation	**Meaning**
HEI	Higher Education Institution
CAS	Course Allocation System
SAD	System Architecture Diagram
ACAS	Automated Course Allocation System

CHAPTER ONE

INTRODUCTION

1.1 Background of The Study

In recent times, technology has constantly become integrated in our daily lives, be it at work or free time and rely on technology in doing different tasks. This change also took place in academic institutions where researchers, teachers and students use this technology to facilitate better learning, increase quality of education as well as to support sustainable practices (Gonzalez Zamar et al., 2020; Kozlova & Pikhart, 2021). Contemporary technologies have dramatically changed the ways of people's communication and interaction in the context of educational settings, e.g., in reference to e-learning platforms, remote work, or automated processes (Kozlova & Pikhart, 2021).

The use of technology in higher education is not only about learning but also improves many other activities on campus. Today, most of the colleges and universities are already focused on furthering the digital transformation of academic processes, especially as we get to the 21st century (Benavides et al., 2020). There is no universal definition of digital transformation but experts have different descriptions of it. Gobble (2018) provides an example; it is explained as a remarkable move for organisations to work i.e. their activities, processes, skills, and structures. It is done with the purpose of using modern technology and its rapidly growing effect on society in a well planned and strategic manner (Benavides et al., 2020).

Digital transformation is the front-runner step organizations make in order to lead change or remain competitive in their industries. In order for HEIs to stay relevant and avoid falling

behind, they need to adopt digital methods. This implies that HEIs should have a clear, organized plan of digital changes that they will adopt in their daily operations and continuously update the plan as technology advances. This is very possible by doing so, allowing them to easily adapt to new trends and keep their position in the education field (Benavides et al., 2020)..

HEIs have incorporated digital technology into areas beyond using e-learning platforms for peer to peer and distance learning. Now, they use publishing platforms that support video, text, or audio content, collaboration tools and communication apps like WhatsApp. Additionally, these technologies enable assessments and give feedback on learning materials. For instance, some universities have begun using virtual and augmented reality to increase collaboration (Pinto & Leite, 2020). Non academic systems such as the registration and application platform are in place to ensure the institution runs efficiently.

With universities striving to develop their workflows into the more digital settings, course allocation is a process that has been getting a fair bit of attention. Simply put, course allocation is assigning a set of courses to a lecturer for a particular semester. Repeated regularly, often every semester or year, this task adjusts to changes in the number of lecturers, courses, classes, or students. The possibility of these changes happening depends on when a lecturer is unavailable for reasons such as maternity leave, retirement or death, or when a course is added or dropped due to shifts in demand or trends (Na & Hussin, 2021).

Higher education institutions in different fields worldwide face a major challenge in course allocation, or academic workloads allocation. Many factors must be taken into consideration to

make effective assignments. This includes the time per course, the number of students enrolled, teaching expertise required, lecturer experience and availability, etc. Institutions can thus design better matches between courses and lecturers, by carefully balancing these factors.

Since it, directly affects students, the allocation of courses is something that matters a lot. It was found through research that students perform better academically and are generally more satisfied with their education if their lecturers are proficient. As well, teaching skills and interpersonal skills are not the only attributes that the lecturer should have in order to help students succeed academically and learn to enjoy the accomplishment of their studies as pointed (Muzenda, 2013; Ng, 2018; Opatha, 2020).

It is important to remember that lecturers have other responsibilities outside simply teaching. Typically, academic work encompasses three areas: teaching, research, and service, which often pertains to duties such as managing administrative work, generating some revenue. Nowadays, all these tasks are distributed evenly so as to prevent overloading a lecturer. However, lecturers have other obligations — they have to support their students, navigate the politics of their university, and continue to work on their careers (Blair, 2019). These added duties can be incredibly time-consuming. A well designed course allocation system ensures that these teaching workloads don’t feel overwhelming to lecturers and that they can get on with their other work obligations.

Methods in statistics and operations research have been used by researchers to create efficient course allocation systems based on the factors and constraints used. For instance, the linear

programming solvers GLPK, COIN-OR CLP, CBC, CPLEX, and GUROBI are some of the studies that have applied for example. Also, others have looked into assignment methods such as the Hungarian method and dynamic programming. For more advanced cases artificial intelligence as well as heuristic approaches such as genetic algorithms and neural networks have been used to optimize the allocation of courses (Hsu, & Chao, 2009; Amuji et al., 2017; Solaja et al., 2020; Na & Hussin, 2021; Soumalias et al., 2022). To investigate the feasibility, the existing solution methods attached to this project will try to simulate this institution by using some of these existing solution methods to create a course allocation system which will allocate courses in an efficient manner.

1.2 Problem Statement

Most HEIs are organized into faculties/colleges composed of departments that offer core and elective courses leading to undergraduate and postgraduate programs (Na & Hussin, 2021). The task of allocating courses to lecturers in the complex hierarchy remains challenging because of continued reliance on manual procedures (Na & Hussin, 2021), despite advances in digital technology. Current approaches are based largely on coordinators' individual judgments rather than disciplined processes (Solaja et al., 2020), consuming a disproportionate amount of time and generating allocations prone to human mistakes, time constraints, and other influences. Group decision-making amplifies these issues through groupthink or cycling of preferences (Coglianese & Lai, 2024), in which consensus takes over from the optimal outcome.

This manual dependency leads to drastic workload imbalances: some lecturers have overfilled teaching loads and others have underutilized capacities. Overworked personnel receive inadequate preparation time, lowering teaching quality and student satisfaction (Amuji et al., 2017; Na & Hussin, 2021). Such imbalances demotivate personnel, increase turnover, and destabilize institutions. Ineffective course allocations can delay graduation of students, forcing HEIs to incur additional resources by way of emergency appointments or remedial courses (Hsu & Chao, 2009; Danner et al., 2018).

To counteract these inefficiencies, this project develops an Automated Course Allocation System (CAS) using Linear Programming optimization. The CAS replaces subjective decision with data-driven decision-making, considering lecturer experience, preference, and institutional constraints to ensure fairness in workload distribution. Processing time is reduced from weeks to minutes through automation, yet the system is responsive to variance in departments through variable parameters. This solution enhances teaching quality, maximizes resource utilization, and makes institutional stability easier through transparent, impartial allocations.

These are aggravated by a deficiency of one-size-fits-all approaches. Workload standards vary across fields (e.g., science lab-based courses and humanities lectures), and departments with less staff typically do not employ official systems of assignment (Danner et al., 2018; Roopchandani, 2024). Instructional loads may be irregular in quantification—by contact hours, credit hours, or ad-hoc units—making additional challenges in attaining parity in allocation. The CAS addresses the issue of heterogeneity by introducing configurable constraint models that respect departmental uniqueness without sacrificing optimization integrity.

1.3 Aim and Objectives of the Study

1.3.1 Aim of the study

The goal of this project is to design an application that optimally assigns courses to lecturers in HEIs.

1.3.2 Objectives of the study

i. Design a system for allocating courses/academic workload to lecturers based on defined criteria

ii. Develop the designed course allocation system to demonstrate the intended proof of concept.

iii. Analyze and evaluate the developed allocation system's performance and capabilities

1.4 Scope and Limitations of the Study

1.4.1 Scope of the study

This project involves designing, developing, and evaluating a web-based course allocation management system specifically for lecturers. This means this project will focus on creating a web-based software solution that assigns courses based on predefined criteria, using insights from existing research. The system's design and features are based on observations and needs identified in the Department of Computer Science at Bells University.

1.4.2 Limitation of the study

The study will focus only on building and testing a course allocation system to see how well it can assign courses efficiently and optimally. This means that other parts of application/software

design (e.g. authentication, user management, and data security), are not considered in this project. Additionally, this project does not involve creating new algorithms or tweaking existing ones for course allocation, but rather using established algorithms to build a comprehensive system.

1.5 Significance of the Study

This project can mean so much and recent events show. For instance, COVID-19 severely affected the global economy, society, and health, closing many institutions including schools and affecting 90 per cent of students worldwide (Erdmann et al., 2021; Marks & Al-Ali, 2022). But this crisis accelerated the move to digital processes, as institutions quickly came to see that digital transformation was a necessity to continue operating. Even after the pandemic was over, lots of organizations wanted to be better prepared for future situations by pushing for automated, digitized operations.

For instance, sub-Saharan Africa is forecasted to account for most of the world's increase in population this century and grow at an annual rate twice the global average forecast from 2020 to 2050 (Gu et al., 2021; O'Sullivan, 2023). This growth is helped more by countries like Nigeria, with some studies saying that actual population increases could be even higher than expected. The big takeaway from these projections? Fast growing countries will eventually have a large, young population — just the one to help boost economic growth if managed well.

JAMB statistics have also shown that this trend exists in exam applicants; registration numbers rose by over 12 per cent between 2022 and 2024 from about 1.5 million to more than 2 million (Tolu-Kolawole, 2024; JAMB Editorial Team, 2022). This then means that HEIs need to build

systems that can scale with increased numbers of students. Replacing manual processes with automated digital methods is one promising solution.

The goal of this project is to digitize the process of allocating courses to lecturers—a task that is currently done manually in many Nigerian universities due to a lack of organized, scalable solutions. Thiis project seeks to allow Bells University of Technology to begin using automated systems in order to optimize their resources, hastening the process of decision making. This solution is also meant to be expandable whereby it could potentially be used to manage other human resources like those of students and non academic staff in the future.

CHAPTER 2

LITERATURE REVIEW

2.1 Introduction

The use of digital technology has been at the center of modern higher education, not just transforming teaching and learning processes but also administrative systems. In today's competitive academic environment, institutions require effective and automated systems that can effectively allocate courses fairly and optimally. This study suggests an Automated Course Allocation System (ACAS) particularly designed for Bells University of Technology to reduce workload by hand and provide a balanced distribution of teaching duties. By applying the latest optimization techniques, the ACAS integrates lecturer preferences, workload limitations, and expertise levels into one system. Not only does this enhance academic timetabling quality, but it also maximizes institutional efficiency as a whole. The system is built on web technologies that promote scalability, responsiveness, and usability, ensuring that all decisions regarding allocation are data-driven and transparent.

Web Applications in HEIs

Web applications have revolutionized higher education institutions (HEIs) by automating thousands of administrative and educational tasks. Initially utilized as a means of complementing online learning platforms, web applications now encompass nearly all areas of education operations—ranging from student registration to course management, faculty scheduling to resource allocation. The incorporation of web technologies within higher education institutions has resulted in systems that not only are accessible and scalable but also highly customizable, to cater to the diverse needs of the current educational environment.

Aside from administrative efficiency, web applications also help universities collect and interpret large datasets for strategic planning purposes. Real-time feedback and analytics of performances for instance facilitate maximum course capacities, enhance instructor quality, and react quickly to trends in student enrollments. The more digital transformations institutions continue to invest in, the greater becomes the imperative role of web applications in enhancing innovations and better education outcomes.

Uses of Web Applications in HEIs

Web applications also serve a number of roles in higher education, both on the academic and administrative sides. On the academic side, they provide robust bases for learning management, virtual classrooms, and the dissemination of digital content. They facilitate seamless interaction among students and teachers, support collaborative learning environments, and allow for flexible, dynamic models of instruction. Web-based systems also offer systems for student assessment and feedback so that learning outcomes are monitored and improved constantly.

From an administration perspective, web applications are key to managing essential functions such as enrollment, course allocation, faculty scheduling, and academic advising. Web applications allow administrators to reduce manual work, eliminate errors, and advocate consistency across institutional processes. Additionally, through the implementation of advanced features such as optimization algorithms and analytics capabilities, web applications allow HEIs to make data-driven decisions that align with both educational policy and budget constraints. In general, the widespread use of web applications across higher education has significantly enhanced operational efficiency and student success throughout the board.

Course Allocation

In the previous chapter, course allocation for lecturers (also referred to as the teaching workload allocation problem) was introduced as a critical process in HEIs to ensure that courses are assigned to lecturers in a way that balances their workload, expertise, and availability. It was also mentioned that proper course allocation is beneficial not only to the lecturers but also to students, as the performance and satisfaction of students is linked to the lecturer's expertise and teaching quality improves when the lecturers have moderate teaching workloads (Saleh et al., 2019). In this literature review, existing course allocation systems, the methods used to assign courses to lecturers, and advanced/novel methods for course allocation will be discussed.

2.2 Key Concepts in Course Allocation

2.2.1 Constraints

Constraints in resource allocation problems are the rules that determine how resources are distributed. These rules typically fall into two categories: hard constraints and soft constraints. Hard constraints are strict requirements that must be met for a solution to be acceptable. For instance, in teaching workload allocation, a hard constraint could be that no lecturer is allowed to teach two classes at the same time—if this rule is broken, the entire schedule becomes invalid.

On the other hand, soft constraints are more like guidelines or preferences that help improve the quality of a solution, but they can be relaxed if necessary. For example, a soft constraint might aim to distribute the teaching load as evenly as possible among faculty members. While it would be ideal for every lecturer to have the same number of classes, the model might allow small differences to ensure the overall solution is practical or to optimize other factors. Essentially, soft

constraints steer the solution toward a more balanced and fair outcome without being as rigid as hard constraints.

Recent research in scheduling and allocation, including studies on university course assignment, uses both types of constraints. For example, Dofadar et al. (2021) developed a hybrid evolutionary algorithm for course allocation that strictly follows hard constraints (like preventing timetable clashes) while using weighted soft constraints to reflect faculty preferences and enhance overall satisfaction. Similarly, in the healthcare field, models for cyclical nurse scheduling also separate non-negotiable hard constraints from more flexible soft constraints to produce efficient and fair work schedules (Saleh et al., 2019b).

2.3 Brief History of Course Allocation in HEIs

The idea of course allocation dates back to the early 20th century, when universities started formalizing their curricula to create standardized learning paths for students. Initially, course enrollment was handled manually, with administrators assigning students to courses based on class sizes and faculty availability (Birnbaum, 1970). As higher education expanded, managing large student populations became challenging, leading to the development of more structured course allocation systems. Early models were rule-based, relying on criteria like student preferences, prerequisite requirements, and faculty workloads (Sönmez & Ünver, 2010).

In the mid-20th century, major universities in the United States and Europe began using mathematical programming models—such as linear programming (LP) and integer programming (IP)—to create formal, algorithm-driven course allocation systems (Dantzig, 1963). During the

1960s and 1970s, optimization techniques were developed to match faculty skills with course requirements while minimizing scheduling conflicts (Roth, 1984).

One of the earliest algorithmic approaches was the Hungarian Method, originally designed for solving assignment problems in operations research (Kuhn, 1955). This method was later applied to course allocation to match lecturers with courses based on factors like teaching load, prerequisite courses, and availability (Budish & Cantillon, 2012). In the 1970s and 1980s, European and North American universities began to implement formal course assignment systems using heuristic and meta-heuristic methods—such as greedy algorithms and simulated annealing—to schedule timetables and assign faculty (Burke, Marecek, & Parkes, 2013). Institutions like MIT and Stanford were pioneers in this area, using linear programming models in their scheduling software (Balinski & Young, 1982).

By the late 1990s and early 2000s, course assignment systems became more advanced with the integration of machine learning algorithms and web-based platforms. These systems allowed for dynamic interaction among staff, students, and the allocation system while incorporating multi-objective optimization to balance factors like staff preferences, class size constraints, and accreditation requirements (Pathak & Sönmez, 2008). Universities such as Harvard and Oxford led the development of structured scheduling frameworks that combined academic advising with enrollment processes, enabling students to register for courses based on priority and departmental availability (Budish, 2011).

Over time, as academic programs grew more complex and computational techniques improved, course allocation methods continued to evolve. The introduction of computerized scheduling in the 1970s and 1980s helped automate student course selection and faculty assignments, reducing

administrative burdens and human errors (Balinski & Young, 1982). The use of database management systems and optimization algorithms in the 1990s further enhanced the management of large-scale course allocations (Sethuraman, Teo, & Tan, 2003). Today, artificial intelligence (AI) and machine learning models allow for real-time adjustments and predictive analytics, making automated course allocation even more effective for decision-making (Budish, 2011).

2.4 Traditional Approaches to Course Allocation

2.4.1 Manual approaches

This approach involves an administrator, course coordinator, or head of department assigning courses to lecturers manually based on factors such as expertise, effectiveness, and availability of the academic staff (Solaja et al., 2020). Unfortunately, these individuals who are responsible for course allocation have no automated tools to optimally assign courses (and are sometimes unaware of such systems) and rely on their intuition and understanding of the lecturer's ability. This is strenuous considering the number of possible courses the department they are in charge of offers (Solaja et al., 2020; Na & Hussin, 2021).

This manual method is difficult and leads to hit-or-miss results, which can lead to many revisions of the course system, making this method inefficient and timewasting and subject to bias judgment (Shohaimay et al., 2016; Na & Hussin, 2021). Even after all this, the final allocations are often times still unsatisfactory, with imbalances in teaching obligations among lecturers,

eventually causing discontent among lecturers or cases of demanding courses being assigned to inexperienced lecturers (Saleh et al., 2019).

2.4.2 Random allocation

Some institutions use tools like spreadsheets (e.g. Microsoft Excel) to assist in allocating courses and rely on random allocations on such spreadsheets to assign academic workloads (Roopchandani, 2024). This is still tedious and arguably worse than the manual method. However, it is made on the assumption that every lecturer is competent enough to handle whatever course he/she is assigned, and is used in situations when there are simply too many teaching staff to consider when allocating courses and the administrator in this case simply doesn't have enough knowledge of his lecturers to allocate courses on discretion.

2.5 Optimization Algorithms for Course Allocation

Optimization algorithms are tools for managing limited resources under various constraints. They first emerged during World War II in the field of operations research, where mathematical models were used to improve military logistics. For example, Dantzig's introduction of linear programming in 1947 marked the start of modern resource allocation methods, while Bellman's development of dynamic programming in the 1950s provided a framework for problems requiring sequential decision-making (Dantzig, 1947; Bellman, 1957). Today, these classical techniques have been extended and refined by recent research to build sophisticated Course Allocation Systems (CAS) for assigning courses to teachers (Ouyang, Li, & Zhou, 2018).

In a CAS, optimization algorithms take into account factors such as a teacher's expertise, availability, workload limits, and institutional rules. Linear programming formulations are often employed to determine the optimal course assignment by balancing competing objectives—such as equalizing workloads or maximizing student satisfaction—while dynamic programming techniques decompose complex allocation problems into simpler subproblems (Li & Zhang, 2019). In cases where the allocation problem is large-scale or NP- hard, heuristic approaches like genetic algorithms provide practical approximations that yield near-optimal solutions in a reasonable time (Ibrahim & Gupta, 2019). In short, these methods enable universities to distribute courses more fairly and efficiently.

One well-known heuristic is the Hungarian method, originally developed by Kuhn in 1955. Recent studies have revisited and enhanced the Hungarian method for modern applications, demonstrating its effectiveness in minimizing assignment costs and balancing performance measures (Chen, Zhao, & Lee, 2021). For instance, its successful implementation in Nigerian universities has shown that applying this method in a CAS can prevent resource waste while ensuring that courses are assigned to teachers whose expertise and workload constraints best match the institutional needs (Ogunleye, Adeyemi, & Musa, 2019).

Other common algorithms include:

- The Greedy Algorithm: assigns courses one after another based on set criteria.
- Linear Programming (LP) and Integer Programming (IP): set up course allocation as a precise mathematical problem.
- Particle Swarm Optimization (PSO): models potential solutions as "particles" that move toward better results.

- Hybrid Algorithms: combine several methods to balance speed and accuracy.

2.5.1 Example linear programming approach to course allocation

In this example linear programming model, we begin by defining two sets: let L be the set of lecturers and C be the set of courses offered during a semester. For each lecturer i (where $i \in L$) and each course j (where $j \in C$), we introduce a decision variable x_{ij}. This variable represents the number of teaching hours that lecturer i is assigned for course j. For example, if x_{ij} is greater than 0, then lecturer i is allocated to teach course j; if it is 0, then they are not assigned to that course.

Each course j requires a specific number of contact hours per week, denoted by S_j, while each lecturer i has a minimum and maximum teaching load—$L_i\text{min}$ and $L_i\text{max}$, respectively. In addition, we capture lecturers' preferences with a parameter p_{ij}, where a higher p_{ij} indicates a stronger preference for lecturer i to teach course j. The goal of the model is to allocate teaching hours in a way that maximizes the overall satisfaction of lecturers by aligning assignments with their preferences.

Mathematically, the model is formulated as follows:

Objective Function:

$$Maximize \sum_{i \in L} \sum_{j \in C} p_{ij} \cdot x_{ij} \quad \textbf{(1)}$$

This function sums the products of the preference weight and the teaching hours over all lecturers and courses.

Subject to Constraints:

Course Coverage: For each course ***j***, the total teaching hours assigned by all lecturers must exactly equal the required contact hours ***Sj***

$$\sum_{i \in L} x_{ij} = S_j, \quad \forall j \in C \tag{2}$$

Lecturer Workload: For each lecturer ***i***, the total assigned teaching hours must be within their allowable workload:

$$L_i^{\min} \leq \sum_{j \in C} x_{ij} \leq L_i^{\max}, \quad \forall i \in L \tag{3}$$

Non-negativity: Each x_{ij} is a non-negative integer:

$$x_{ij} \in \mathbb{N}_0, \quad \forall i \in L, \ \forall j \in C \tag{4}$$

Here, Equation (**1**) is the objective that aims to maximize the sum of preference scores, while Equations (**2**) and (**3**) ensure that each course is completely covered and that each lecturer's total workload stays within their assigned limits. The non-negativity and integrality condition

(Equation (**4**)) ensures that teaching hours are whole numbers, which reflects real-world scheduling where fractional hours are not practical. (Shohaimay & Ujang, 2024)

Recent studies have effectively applied integer linear programming (ILP) models to address teaching load allocation in academic settings, achieving fair and efficient distributions while incorporating various constraints to reflect institutional requirements. (Caselli et al., 2020)

For instance, a study developed and validated an ILP model to optimize lecturer-to-course assignments by considering workload-related constraints such as class capacity, course contact hours, course credits, and the number of courses per lecturer. The model demonstrated robustness and efficiency in balancing workloads, reducing biases, and ensuring a fair assignment approach (Rakgomo et al., 2023).

Another case study proposed an ILP model to optimize academic timetables, focusing on course scheduling and faculty availability. By incorporating a minor structural adjustment to enhance computational efficiency, the model generated optimal scheduling solutions within minutes, even for large-scale problems involving over 200 classes and 100 lecturers. The approach effectively minimized planning time, identified unavoidable scheduling conflicts, and highlighted unschedulable classes due to constraint violations (Talmor, 2024).

Additionally, mixed-integer linear programming models have been developed for assigning teaching assistants to tutorials, aiming to maximize the number of tutorials taught by the most suitable teaching assistants. These models account for different capabilities of teaching assistants, their maximum teaching loads, and potential scheduling conflicts, thereby improving teaching quality and efficiency (Qu et al., 2017).

2.6 Related works

In "An Algorithm for Efficient Allocation of Courses to Lecturers for Effective Teaching," Anichebe G.E. presents a formal, rule-based algorithm to automate course-to-lecturer assignment in Higher Education Institutions. His work is prompted by the need to substitute intuition-driven, ad-hoc assignments with a transparent, reproducible process that determines teaching capacity and specialization match.

Anichebe's long-term aspiration is to offer a quantitative approach that ensures equity and efficiency in course allocation. By way of systematic ranking of lecturers against multiple pedagogical and professional attributes, the algorithm eliminates judgments in hand assignments—such as cronyism or laziness—yet respects institutional workload and area of specialization constraints.

In order to achieve this, the paper proposes three particular objectives. First and foremost, it tries to establish an active rating system that can blend various lecturer characteristics—rank, experience, and performance—into a blended combined rating. Secondary to that, it tries to establish workload caps so that no lecturer teaches more than three classes in a semester. Thirdly, it tries to distribute equal attention to all courses by balancing senior and junior lecturers wherever necessary. Methodologically, Anichebe employs an observation approach for collecting secondary data on lecturers' performance from institutional reports and web-based appraisals. The algorithm follows step by step: computing scaling factors for each attribute, aggregating them into a total percentage score, ranking candidates, and allocating courses iteratively subject to the given constraints.

The ranking itself is based on consideration of five basic variables: lecturers' ranking, years of teaching, specialization by subject, performance score of the students, and purposive senior/junior faculty pairing. These variables are weighted and transformed into a linear scale before being inputted into the end rating to provide a multidimensional measure of lecturer fit in the course.

2.6.1 Mathematical Representation

A unique feature of his work is the pairing of two or more lecturers to a course, usually a senior and a junior. The senior lecturer is responsible for 60% of the course content, while the junior handles the remaining 40%, formally represented as:

$$\text{Load}_{\text{senior}} = 0.6 \times C, \quad \text{Load}_{\text{junior}} = 0.4 \times C \tag{5}$$

Where C is the total course workload. This not only ensures better teaching coverage but also promotes mentorship and skill development among junior staff. The algorithm further caps course assignments per lecturer by enforcing:

$$c_i \leq 3, \quad \forall i \in \{\text{lecturers}\} \tag{6}$$

Where c_i denotes the number of courses assigned to lecturer i.

Anichebe's algorithm can be formalized as follows. Let:

$$S = \bigcup_{i=1}^{n} S_i$$

- be the set of all subject areas offered by a department, partitioned into subsets S_i corresponding to areas of specialization, such that:

$$S_1 \cup S_2 \cup \cdots \cup S_n = S, \quad S_1 \cap S_2 \cap \cdots \cap S_n = \emptyset \tag{7}$$

- For each lecturer i, define a set of selected specializations $\sigma i \subseteq S$ with $|\sigma i| \geq 2$. A lecturer is considered eligible for a course k if:

$$\sigma_i \cap \{\text{subject area of } k\} \neq \emptyset \tag{8}$$

Next, assign scaling factors for each lecturer i:

- r_i: rank factor (e.g., 0.6 for senior, 0.4 for junior)
- e_i: experience factor (e.g., 0.45 for >=10 years)
- p_i: performance factor (normalized from 1%–100% into 0.01–1.00)

The overall score O_i is the mean of these factors:

$$O_i = \frac{r_i + e_i + p_i}{3} \tag{9}$$

In the sample computation for three senior lecturers (John, Mary, Andrew) in Software Engineering:

$$\begin{aligned} O_{\text{John}} &= \frac{0.6 + 0.35 + 0.71}{3} = 0.5533, \\ O_{\text{Mary}} &= \frac{0.6 + 0.25 + 0.74}{3} = 0.5300, \\ O_{\text{Andrew}} &= \frac{0.6 + 0.4 + 0.62}{3} = 0.5400. \end{aligned} \tag{10}$$

These scores are then ranked in descending order to determine priority for course allocation.

The pseudocode is summarized algebraically:

- For each course ***k***:
    - Define the senior candidate set $S_k = \{i : i \text{ is senior and } \sigma_i \cap k \neq \emptyset\}$
    - Select $s = \arg\max_{i \in S_k,\, c_i < 3} O_i$
    - Assign course to ***k*** to ***s***, increment c_s.
    - Repeat for junior set J_k to find ***j***.
- Ensure all lecturers receive at least one course by reiterating assignments for any ***i*** where $c_i = 0$.

His goal was to maximize teaching effectiveness by assigning each course to the highest-scoring, available lecturers, while respecting workload limits and promoting collaboration through senior–junior pairing.

2.6.2 Relation to Proposed Project

Project CAS uses these factor-based principles directly by placing them in a Linear Programming (LP) framework. We retain Anichebe's core assessment criteria—rank of lecturer, experience, specialty, and performance—but define them as LP variables and constraints so that they can be globally optimized for assignments rather than isolated, per-course choices.

One of the main differences is implementation: while Anichebe's work remains at the pseudocode and idea stage, Project CAS delivers a functional, web-based and desktop application. Leverage of React for front-end, Node.js for back-end, and Electron for desktop

deployment, our solution translates the theoretical algorithm into an intuitive system with dashboards, reporting, and live updates.

2.6.3 Contrast From Existing Work

As far as the differences in algorithmic models go, Anichebe takes a greedy sequential course assignment loop—i.e., assigning the highest ranked available instructor to a course sequentially—whereas Project CAS formulates the assignment as an LP problem once globally solved in its entirety. Such a switch in strategy from myopic to global optimization is designed to achieve maximum global balance of workload and preference overall across the lecturer pool in total. In addition to algorithmic improvement, Project CAS incorporates a Waterfall-Agile hybrid approach with systematic planning phases combined with successive sprints. This allows for thorough UI/UX testing, modular code design, and straightforward incorporation of stakeholder feedback—topics not addressed in the first algorithmic framework.

Table 2.1 Related works

Author/Year	Methods/ Methodologies	Results	Limitations/ Drawbacks
Muzenda (2013)	Qualitative research, analyzing lecturer competencies and their impact on students' academic performance.	Discovered that lecturer expertise, teaching ability and interpersonal skills greatly affect student satisfaction and academic outcomes.	Limited to specific competencies, not accounting for contextual factors, e.g, cultural differences.
Anichebe G.E (2018)	An algorithm for efficient course allocation to lecturers to improve teaching quality in Nigerian tertiary institutions.	Encourages fair course distribution and improves teaching quality in HEIs	Requires frequent data updates, and may cause conflicts in paired teaching
Gobble (2018)	Literature review focusing on the concept of digital transformation and its application in various sectors, including education.	Defined digital transformation as a major organizational shift that utilizes modern technologies to enhance process and competitiveness.	Lacked empirical validation for claims related to digital transformation outcomes.
Ng (2018)	Survey-based study analyzing the competency of lecturers and its impact on student satisfaction.	Verified that lecturer expertise positively influences student satisfaction with the learning experience.	Limited scope, as it only focused on student satisfaction,ignoring other academic performance indicators.
Benavides et al (2020)	Systematic literature review on on digital transformation in HEIs (Higher Education Institutions).	Pinpointed trends and challenges in HEIs' adoption of digital transformation strategies.	No specific solutions were provided to overcome highlighted challenges.
Opatha (2020)	Analyzed how	Discovered that	Limited

	lecturer competency influences student satisfaction through mediating variables like teaching behaviours.	teaching behaviours mediate the relationship between lecturers' competencies and student satisfaction.	generalizability as the study only focused on specific cultural contexts.
Solaja et al (2020)	Used the Hungarian method (assignment problem) to optimize resource allocation in Nigerian institutions.	Demonstrated that the Hungarian method adequately minimizes resource wastage during course allocation.	Focused only on the Hungarian method, without comparing alternative methods.
González-Za mar et al (2020)	Assessed the role of ICT in sustainable education using data-driven approaches.	Highlighted the importance of ICT for improving sustainable education practices in higher education.	Lack of experimental validation for proposed ICT strategies.
Erdmann et al (2021)	Analyzed the impact of COVID-19 on digital transformation in universities using surveys and case studies.	Concluded that the pandemic accelerated the adoption of digital tools in education.	Limited to short term impacts, paying no attention to long-term implications of accelerated digitization.
Na & Hussin (2021)	Made use of Python-based algorithms for implementing course allocation in HEIs.	Improved the efficiency and transparency of the course allocation process.	Limited evaluation metrics, focused mainly on technical performance.
Kozlova & Pikhart (2021)	Survey of University students on their view of ICT usage in Higher Education.	Discovered that students see ICT as crucial in enhancing the quality of learning in higher education.	Limited to self-reported perceptions without analyzing actual outcomes.
Soumalias et al (2022)	Researched on the application of Machine Learning in optimizing course allocation.	Showed that Machine Learning could dynamically adapt to changing course allocation requirements.	Complex implementation, resource-intensivene ss limit feasibility for widespread adoption.

Marks & Al-Ali (2022)	Proposed a framework to assess the maturity of digital transformation in HEIs.	Provided a systematic approach to evaluate digital readiness in higher education.	Effectiveness of the framework was not tested in diverse institutional settings.
O'Sullivan (2023)	Demographic analysis of global population growth trends and their implications for sustainable development.	Identified that fast-growing populations like that of Nigeria present opportunities for economic and educational growth if managed effectively.	Projections may not account for unforeseen socio-economic variables.
Bullock et al (2024)	Reviewed group decision-making challenges in AI governance, focused on biases and inefficiencies in collective decision making.	Highlighted biases like Groupthink and preference cycling as barriers to effective decision making in organizations.	Limited focus on group decision making without exploring individual-level biases.

Overall, while Anichebe et al. (2018) gives us the underlying principle of scoring and assignment for lecturer-course assignments, Project CAS makes it one notch higher by achieving it in an LP-formulated, production-level context. Our system not only simplifies the assignment process, but it also provides administrators flexibility in terms of parameter modification, viewing instantaneous resolution of conflict, and scaling to large-scale complex datasets—further extending the scope and magnitude of the original algorithm.

2.7 Advanced Approaches for Course Allocation System

The usage of computer systems to assign courses to instructors has come a long way from old-fashioned scheduling methods. Today, advanced mathematical and computational techniques, like multi-objective optimization, combinatorial optimization, and graph theory (Amuji et al. 2017; Solaja et al. 2020), are used to make course assignments more efficient, fair, and flexible.

These modern systems can quickly adjust lecturer workloads, make the best use of available resources, and even resolve scheduling conflicts automatically (Hsu & Chao, 2009). In addition, various methods such as Pareto efficiency, game theory, and linear programming provide clear, systematic ways to design robust allocation systems that follow institutional rules and respect professors' preferences (Benavides et al., 2020). As a result, these innovations help create scalable systems that improve academic planning and support better decision-making in higher education institutions (Soumalias et al., 2022).

2.7.1 Multi-objective optimization

Multi-Objective Optimization (MOO) is a method that helps solve problems with several conflicting goals at the same time. Unlike single-objective optimization, which focuses on maximizing or minimizing one factor, MOO balances multiple criteria at once (Amuji et al., 2017). In an automatic course scheduling system for lecturers, MOO makes sure that various aspects—like balancing workloads, matching lecturer expertise, meeting schedule constraints, and following institutional rules—are all considered. Essentially, it finds a compromise that works well for everyone involved, promoting fairness and efficiency (Hsu & Chao, 2009).

The idea behind multi-objective optimization goes back to the mid-20th century, with key contributions from Vilfredo Pareto, who introduced the concept of Pareto efficiency. Pareto efficiency is an economic principle that states resources are allocated in the best possible way when it isn't possible to improve one area without making another worse. This concept is widely used in economics, decision-making, and resource allocation, including in course assignment in Higher Education Institutions (HEIs) (Benavides et al., 2020). Over time,

methods like goal programming, evolutionary algorithms, and artificial intelligence have further developed MOO (Charnes & Cooper, 1961; Hsu & Chao, 2009).

In practice, MOO works by setting multiple goals and using techniques to balance them. For instance, the Weighted Sum Method assigns a weight to each goal based on its importance, while Pareto Optimization generates several optimal solutions where improving one goal would mean sacrificing another (Solaja et al., 2020). In the context of course allocation, an MOO-based system takes into account factors such as lecturer preferences, available time slots, course

demand, and workload limits. The system then produces a schedule that minimizes conflicts and distributes teaching loads fairly (Soumalias et al., 2022).

A practical example of MOO in course allocation can be seen in Nigerian universities, where challenges like staff shortages and scheduling conflicts are common (Amuji et al., 2017). For instance, if a department has 20 lecturers and 50 courses to assign, an MOO-based system would consider each lecturer's area of expertise, preferred teaching times, and other university constraints to create a balanced schedule. This approach ensures that no lecturer is overburdened while also maximizing course coverage and improving student satisfaction.

2.7.2 Heuristic and meta-heuristic techniques

Heuristic and meta-heuristic methods are used to find approximate solutions to tough optimization problems when getting an exact answer is too slow or not practical. In simple terms, heuristics rely on specific strategies from a given field to quickly come up with a solution that works, even if it isn't perfect. Meta-heuristics, on the other hand, provide a broader framework that guides these heuristic methods to explore more of the solution space. This combination is especially useful in automated course allocation, where factors like workload balance, matching lecturer expertise, and schedule availability must all be considered (Hsu & Chao, 2009).

The idea of using heuristics for problem-solving goes back to George Pólya in the mid-20th century. Later, meta-heuristic methods were introduced, with key examples being Holland's Genetic Algorithms in 1975 and Kirkpatrick et al.'s Simulated Annealing in 1983. Over time, these approaches have been applied to a variety of challenges, including resource allocation and scheduling in higher education (Solaja et al., 2020).

In practice, simple heuristic methods like Constraint Satisfaction Methods and Greedy Algorithms are used to quickly generate a workable course schedule. These initial solutions are then refined using meta-heuristic techniques such as Genetic Algorithms, Particle Swarm Optimization, and Tabu Search, which improve the schedule step by step according to set criteria (Soumalias et al., 2022). These methods are very effective in handling the large number of variables and constraints involved in course scheduling.

Because universities have to juggle many faculty members and scheduling restrictions, heuristic and meta-heuristic approaches are often the best way to solve course allocation problems. These strategies help balance institutional rules, class availability, and lecturer preferences. By using these methods, educational institutions can create fairer and more efficient course schedules, reduce conflicts, and ensure a balanced distribution of teaching responsibilities (Amuji et al., 2017).

In summary, combining optimization techniques with heuristic and meta-heuristic methods makes automated course allocation systems more efficient, equitable, and flexible. This mix

of approaches supports smarter decision-making and helps build robust, scalable systems that meet the needs of both lecturers and the institutions they serve.

2.8 Typical Operation of a CAS

A typical course allocation system for lecturers operates with the following steps explained below as shown in Figure **2.1**;

Data Collection: The system gathers information about faculty, including their class preferences, availability, and areas of expertise. It also collects institutional constraints like maximum teaching loads and fixed class time slots.

Execution of Optimization Algorithm: The collected data is fed into advanced multi-objective optimization models. These models work to balance factors such as faculty workload, student demand, and administrative limits.

Conflict Prevention: The system identifies and fixes scheduling conflicts—like overlapping classes or too many teaching hours—by generating alternative allocation options.

Automated Schedule Generation: Once the optimal schedule is determined, it is approved and shared with both students and faculty through a central web-based platform.

Real-time Adjustments: If a faculty member becomes unavailable, course demand changes, or any other adjustments are needed, the system automatically updates and reassigns the schedule in real time (Carnegie Mellon University, 2010).

Generally, the following is the formal workflow of modern course allocation systems:

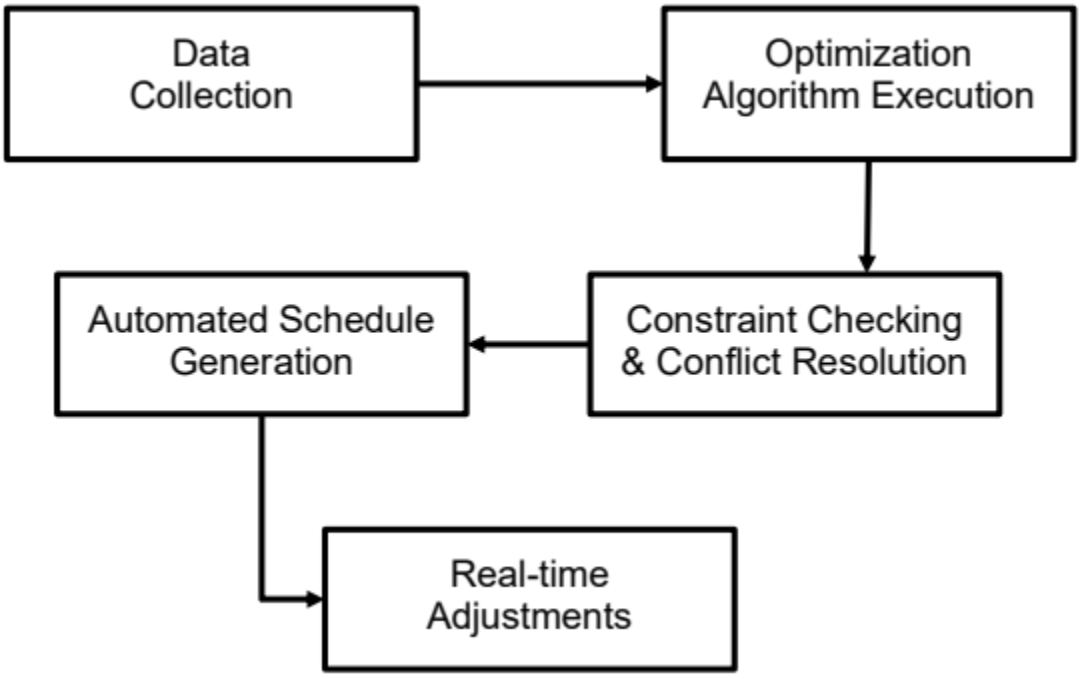


Fig 2.1 Workflow structure of a typical Course Allocation System

2.9 Evaluation of CAS in HEIs

2.9.1 Simulation and scenario testing

Simulations and scenario testing are key to automating course allocation because they allow institutions to try out different allocation strategies before fully implementing them. Essentially, these methods create a virtual version of the course allocation system so administrators can see how factors—such as lecturer workloads, scheduling limits, or course popularity—affect the final assignments (Solaja et al., 2020). By running different scenarios, institutions can spot potential bottlenecks, forecast system performance, and tweak the allocation process to be more effective and fair.

The use of simulation in university timetabling goes back to the early days of computer modeling in operations research in the mid-20th century. Since then, simulation techniques have advanced, incorporating methods like discrete event simulation and Monte Carlo techniques. These advanced methods allow for probabilistic evaluations across different allocation scenarios, enabling universities to test and refine their models before putting them into action (Benavides et al., 2020).

In practice, simulation models use real-world data—such as professor preferences, student enrollment figures, and institutional policies—to generate a range of possible allocation options. Each scenario is then evaluated using predefined metrics like workload balance, scheduling conflicts, and adherence to policies. Decision-makers can iterate through these scenarios to find the best allocation approach and reduce the risk of disruptions (Soumalias et al., 2022).

Scenario testing is also widely used in universities to check how well automated course allocation systems perform under different conditions. For example, a university might simulate a sudden increase in student enrollments to see if its current allocation method can handle the extra demand without overloading professors. Scenario testing can also highlight potential scheduling conflicts among lecturers and suggest ways to resolve them (Amuji et al., 2017).

2.10 Existing Implementations/Case Studies of CAS in HEIs

At the University of California, Berkeley, an AI-supported workload optimization platform is used to assign teaching duties based on faculty expertise. The system considers factors such as specialization, availability, and past teaching records to ensure that each course is handled by the most qualified instructor (Amuji et al., 2017).

The London School of Economics (LSE) synchronizes lecturer schedules with student enrollment trends through a demand-aligned faculty allocation system. This approach helps prevent any faculty member from being overloaded while still meeting the student demand for popular subjects (Soumalias et al., 2022).

At the Technical University of Munich (TUM), an automated course allocation framework is in place that takes both teaching and research responsibilities into account. This system ensures that faculty have enough time for research while also fulfilling their teaching duties (Benavides et al., 2020).

Meanwhile, the University of Sydney uses a responsive scheduling system that adjusts course assignments in real-time based on staffing changes. This means that even if there are unexpected absences or departmental reorganizations, teaching continuity is maintained (Solaja et al., 2020).

Course Allocation Systems in Nigerian HEIs

A Dynamic Programming–Based Course Allocation Model was developed by Amuji et al. (2017) at the Federal University of Technology, Owerri. This model uses a multi‑stage dynamic programming approach to optimally allocate courses among lecturers so that each lecturer receives an ideal credit hour load (e.g. 12 credit hours per semester).

The Nigeria Army College of Environmental Science and Technology (NACEST) designed an Automated Course Allocation System for Lecturers. It automates the process of assigning courses to lecturers. It addresses issues such as inaccurate manual assignments and ensures that lecturers are allocated courses matching their areas of expertise.

The Department of Computer Science at the University of Ilorin also implemented an Automated Online Course Allocation Management System that streamlines course assignment by allowing administrators to manage course–lecturer matches digitally. It factors in lecturers' expertise, rank, and workload to produce more balanced schedules.

2.11 Other Applications of Allocation Algorithms

2.11.1 Student admission allocation

Student admission allocation is the process by which higher education institutions decide how many students to admit into various programs based on criteria like academic merit, entrance exam scores, quotas, and the institution's capacity. The main goal is to balance access and quality by placing students in programs where they have the best chance to succeed while keeping the institution’s resources well-managed (Marginson, 2018).

In the past, admission decisions were largely made through a discretionary process where applications were manually reviewed and decisions were based on subjective judgments. Over time, universities moved toward a more structured system by introducing standardized entrance exams—such as the SAT in the United States, which started in 1926, and Nigeria's JAMB, established in 1978. This shift helped adopt merit-based admission policies to reduce bias and ensure fairness for all applicants (Altbach, 2016).

Today, many institutions have started using automated admission systems powered by AI and data analytics to predict student success better, optimize enrollment, and boost student diversity. According to Varghese (2017), these modern systems evaluate factors like academic performance, extracurricular achievements, and socioeconomic background. Looking ahead, the use of predictive modeling and machine learning in admissions will likely increase, helping institutions identify which students are most likely to succeed and creating personalized admission pathways (Marginson, 2018).

2.11.2 Faculty and staff allocation

Faculty and Staff Allocation refers to how higher education institutions decide on the number of staff members and assign them to various colleges, departments, research projects, and support roles based on criteria like expertise. In essence, it's about matching academic and administrative staff with the areas where they are most qualified and needed. This process is crucial because it ensures that workloads are fair, instructors teach in their areas of specialization, and the institution's resources are aligned with its academic and research goals (Teichler, 2003).

A poorly designed allocation system can lead to issues such as burnout, inefficiency, and lower academic performance. In contrast, a well-organized system can enhance teaching quality, boost job satisfaction among faculty, and improve overall institutional efficiency. The practice of faculty distribution dates back to medieval universities in the 12th and 13th centuries, where teaching roles were assigned informally based on factors like age, status, and expertise (Rashdall, 1895). Back then, many academics, often clerics or monks, handled a mix of academic and administrative duties without a clear system. As colleges grew in the 19th and 20th centuries, the process became more structured, and the introduction of tenure systems in the early 20th century helped institutionalize long-term job security, influencing faculty retention and workload policies (Altbach, 2016).

By the mid-20th century, government policies and funding began to significantly influence faculty allocation. Universities started to implement structured workload policies that clearly defined faculty responsibilities across teaching, research, and administrative tasks (Varghese, 2017). With a greater focus on research output and increasing student enrollments, institutions adopted a data-driven approach to faculty distribution, ensuring that teaching

assignments matched both institutional goals and available funding. Today, universities use digital workforce management systems, workload management tools, and faculty performance analytics to streamline resource distribution. AI-based models now even forecast staffing needs, track workloads in real time, and recommend the best course assignments based on each faculty member's expertise and the institution's priorities (Teichler, 2003).

2.11.3 Classroom and Facility Allocation

Classroom and facility allocation refers to the systematic assignment of physical spaces within an institution to maximize their use for academic and administrative functions. This process involves organizing lecture halls, laboratories, offices, and other essential facilities based on the institution's needs, faculty schedules, and enrollment data. Effective allocation minimizes scheduling conflicts, makes the best use of available resources, and creates a supportive learning environment for both students and instructors (Burke & Petrovic, 2002).

The idea of organized classroom and facility allocation emerged during the rapid expansion of large educational institutions in the late 19th and early 20th centuries. As universities grew in size and complexity, manual scheduling methods became insufficient, leading to the development of early timetable management strategies. Frederick Taylor's work in scientific management emphasized the importance of resource optimization, laying the foundation for structured space allocation in education (Taylor, 1911). By the mid-20th century, institutions began using basic scheduling algorithms to assign classrooms based on time availability, faculty timetables, and student enrollment numbers (Burke et al., 2004). Over time, the introduction of computerized scheduling models significantly improved efficiency, enabling institutions to

manage larger student populations and a broader range of course offerings (Burke & Petrovic, 2002).

Today, classroom and facility allocation in higher education relies on specialized scheduling systems that consider constraints such as seating capacity, faculty schedules, and curriculum requirements. These systems employ optimization algorithms to balance demand and supply, ensuring that classrooms are assigned efficiently while reducing conflicts. Many modern systems also include dynamic rescheduling features, allowing administrators to quickly adjust allocations in response to unforeseen changes like faculty absences or shifts in student enrollment (Burke et al., 2004).

In recent years, the process of classroom and facility allocation has become largely automated through the use of artificial intelligence, big data analytics, and cloud-based technologies. Universities around the world now utilize advanced scheduling software to optimize space utilization, lessen administrative burdens, and enhance the overall learning experience. These advancements enable institutions to manage their resources more effectively, accommodate increasing student numbers, and ensure that academic spaces are used to their fullest potential (Burke & Petrovic, 2002).

2.11.4 Financial resource allocation

Financial resource allocation involves strategically assigning financial resources within an institution to support operations, educational programs, infrastructure, and research activities. This approach ensures that funds are distributed effectively to address institutional priorities,

maximize resource use, and maintain fiscal sustainability. When done well, effective financial allocation helps institutions achieve long-term goals, control costs, and provide essential support for faculty, students, and staff (Johnstone & Ireson, 2005).

The concept of financial resource allocation has its roots in the economic theories of the 18th century, notably influenced by Adam Smith's ideas on resource distribution and public finance. Over time, governments and institutions began formalizing their budgeting processes to secure a fair distribution of funds for public services, including education. In the 20th century, colleges and universities adopted systematic budgeting approaches—such as line- item budgeting and performance-based funding—to manage financial resources more efficiently (Tight, 2000). Later, universities integrated financial planning frameworks that combined economic forecasting, assessments of institutional needs, and cost-benefit analyses to further improve resource distribution (Baker, 2005).

In higher education institutions, financial resource allocation is managed through structured budgeting systems that consider factors like tuition revenue, government funding, endowments, and research grants. Institutions use methods such as incremental budgeting, zero-based budgeting, and priority-based funding to ensure that resources align with academic and administrative objectives. Budget committees and finance professionals review expected expenses, departmental needs, and institutional goals to craft an optimal allocation strategy that supports teaching, research, and infrastructure development (Kezar, 2014).

Today, financial resource allocation in HEIs has advanced through the use of data analytics, financial modeling, and automated budgeting tools. Universities now employ digital financial management systems to monitor expenses, forecast funding needs, and ensure transparency in

resource distribution. Moreover, performance-based funding strategies link financial support to student outcomes, faculty productivity, and overall institutional effectiveness. As financial management practices in higher education continue to evolve, strategic allocation remains a crucial element in ensuring operational excellence and promoting academic advancement (Scott & Kezar, 2015).

CHAPTER THREE

METHODOLGY

3.1 Introduction

This chapter describes the methods used to design and build the Automated Course Allocation System (CAS) for lecturers. It explains the development process, including the selection of technologies, the creation of the allocation algorithm using Linear Programming, and the evaluation methods in testing.

Diagrams like flowcharts, use case diagrams, and sequence diagrams are used to illustrate the system architecture and data flow. The system's performance is measured by its accuracy, efficiency, and fairness. A testing phase, with documented procedures and test inputs, confirms that the system meets its objectives through proper design, algorithms, and testing.

3.2 System Development Process

This section outlines the methodology used and tools applied in developing the Automated Course Allocation System (CAS) for instructors. The development process involved strategic planning and the application of a hybrid approach that combines elements of both Agile and Waterfall. This approach was chosen to balance structured planning with adaptive development to facilitate effective development of the system with some flexibility for realignment as required. Because the project is being built standalone with minimal real-time stakeholder feedback, this methodology offers a stable foundation while leaving space for incremental growth.

The methodology puts emphasis on careful upfront planning in order to eliminate as much development risk as possible, but incorporates Agile's iterative method so that each piece is tested and refined as it is developed. A variety of tools and technologies were also used in an effort to enhance the system's functionality, performance, and scalability, from the Front end and Back end to the database and optimization algorithms applied to allow for automated allocation. VS Code, GitHub, Postman, and Vercel as support tools were also employed to facilitate development, version control creation, and simplify deployment and testing. This methodology and tool set ensures the end system meets performance criteria as well as satisfies lecturers' and administrators' needs.

The project will use a hybrid approach that combines elements of both Agile and Waterfall methods as shown in Figure 3.1.

3.2.1 Development methodology

In this project, we chose a Waterfall-Agile Hybrid approach to build the Automated Course Allocation System (CAS) because it combines the strong planning of the Waterfall model with the flexibility of Agile methods. The Waterfall part lets us define all major requirements—like the Linear Programming algorithm, database design, and system workflow—before coding starts. This careful upfront planning helps reduce uncertainty and prevents major issues later on.

At the same time, the Agile component allows us to develop the system in small, manageable phases. Each module is built, tested, and refined in iterations, so we can catch and fix problems early and adapt to any changes that come up during development. This mix of thorough planning

and ongoing feedback makes the CAS both stable and adaptable, ensuring it meets the project needs effectively.

3.2.2 Development phases

i. Planning: In this stage, we will define the project goals and gather detailed requirements from lecturers and administrative staff. Since we expect minimal feedback during development, it is important to identify all requirements early. This planning will ensure that the system's functions meet user needs and follow institutional policies.

ii. Design: At this stage, we will design the system's architecture and user interface using diagrams and wireframes. These documents will serve as clear guides for both frontend and backend development, helping to keep the project on track and aligned with user requirements.

iii. Development: The system will be developed in small steps. We will code and test each core function one at a time before moving on to the next. This modular approach will allow us to monitor progress closely and address any issues early on. For example, we will develop features like course data management, lecturer profiles, and the allocation algorithm incrementally.

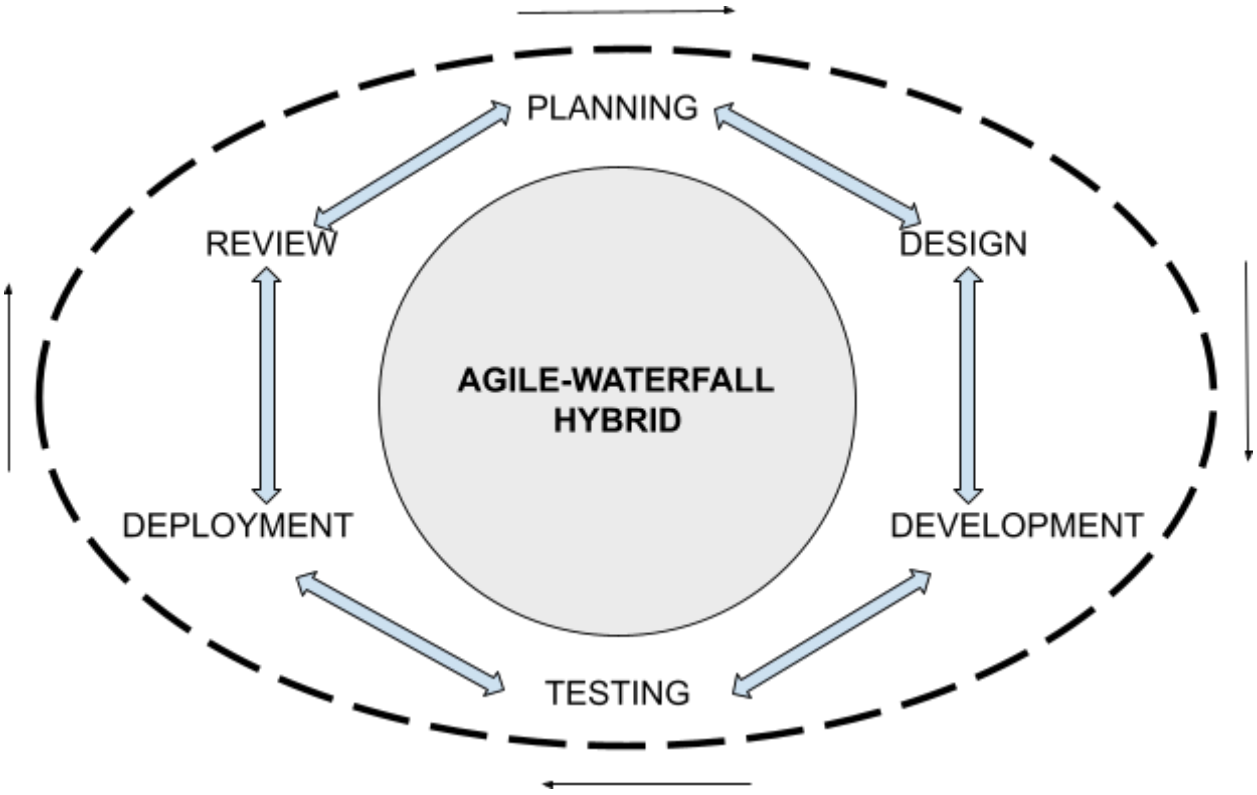


Fig 3.1 Agile-Waterfall Hybrid Approach

iv. Testing: Testing will be conducted alongside development to verify that each component works as intended. After the core system is complete, we will carry out end-to-end testing to evaluate performance, reliability, and scalability. This phase will ensure that the system can handle real-world scenarios and user interactions.

v. Deployment: Once the core functions have been implemented and tested, we will deploy the application. At this stage, we expect every component to function correctly in the production environment and meet the acceptance criteria.

vi. Review & Maintenance: After deployment, we will collect user feedback to guide future revisions or improvements. This stage will allow us to adjust the system based on actual usage, ensuring that it remains effective and useful.

3.3 Tools and Technologies

3.3.1 Frontend development

i. React.js: React is a widely used JavaScript library for building user interfaces, especially for single-page applications that update data without reloading the page. Its component- based structure lets developers create reusable parts, making the code more modular and easier to maintain. React's virtual DOM improves performance by updating only the parts of the page that change, which creates a more responsive interface. For the CAS, React helps build a dynamic and interactive interface so that lecturers and administrative staff can easily use the system.

ii. Tailwind CSS: Tailwind CSS is a utility-first CSS framework that uses simple utility classes to style custom layouts directly in the HTML. This approach gives developers

control and flexibility when styling, allowing them to build responsive interfaces quickly. With Tailwind CSS, CAS can keep a consistent design across all components, ensuring a better user experience and reducing the need for custom CSS.

3.3.2 Backend development

i. Node.js: Node.js is a JavaScript runtime that uses Chrome's V8 engine to run code on the server. Its event-driven and non-blocking design makes it a good choice for building scalable and efficient server applications. In the CAS, Node.js manages server logic, processes requests, handles data operations, and runs the optimization algorithms needed for course allocation.

ii. SQL: SQL is the standard language used for managing and querying databases. For the CAS, we are using a lightweight SQL database that works well with Electron. This database is ideal for offline applications, allowing the system to store and manage data even when it is not connected to the internet.

3.3.3 Algorithm and deployment tools

i. JavaScript LP Algorithms: Linear Programming (LP) is a method used to find the best outcome in an optimization problem. Using JavaScript to implement LP algorithms lets us easily connect the frontend and backend. In a Node.js environment, LP solvers can run directly, helping with tasks like finding the optimal course allocation. There are several JavaScript libraries available for this purpose, such as glpk.js and javascript-lp-solver, which help perform the necessary calculations efficiently and reduce development effort.

35ii. Electron: Electron is a framework that allows us to build desktop applications using web technologies like JavaScript, HTML, and CSS. By using Electron, we can package the CAS as a

self-contained application that runs on multiple operating systems. This means that users can install and use the system offline, ensuring a consistent and reliable experience.

3.3.4 Development tools

i. Visual Studio Code (VSC): This is a free and open-source code editor developed by Microsoft. It supports many programming languages and includes features like debugging, version control, and code completion, which help improve developer productivity.

ii. GitHub: This platform is used for version control and collaborative development. GitHub makes it easy to track changes, manage code versions, and work together on the project.

3.4 System Workflow

As shown in the flowchart Figure 3.2 and sequence diagram in Figure 3.3, the system workflow consists of the following:

I. **Data Input**: The system collects key information including:

i. Course Data: Names, credit units, codes, and prerequisites.

ii. Lecturer Data: Names, specializations, maximum workload, and availability.

iii. Constraints: Department rules, course prerequisites, and lecturer availability.

II. **Data Validation**: The system checks the data for missing information, incorrect formats, and conflicts. If there are errors, the user must correct the data.

III. **Preprocessing (Mapping & Structuring)**: Validated data is mapped to suitable lecturers based on specialization, workload, and priority to ensure the algorithm runs effectively.

IV. **Algorithm Execution**: The Linear Programming (LP) model is executed:

i. Decision variables set course-to-lecturer assignments.

ii. The objective function minimizes workload imbalances.

iii. Constraints ensure proper allocation.

V. **Conflict Resolution**: The system resolves issues such as overloaded lecturers or unassigned courses. Unresolved conflicts are reported as errors

VI. **Display Results**: The allocation results are shown in a table for administrative review.

VII. **Admin Review and Confirmation**: Administrators review the results and can adjust them if necessary.

VIII. **Database Update**: Once confirmed, the results are securely updated in the database.

IX. **Report Generation**: Optional reports summarize workload distribution, efficiency, and errors.

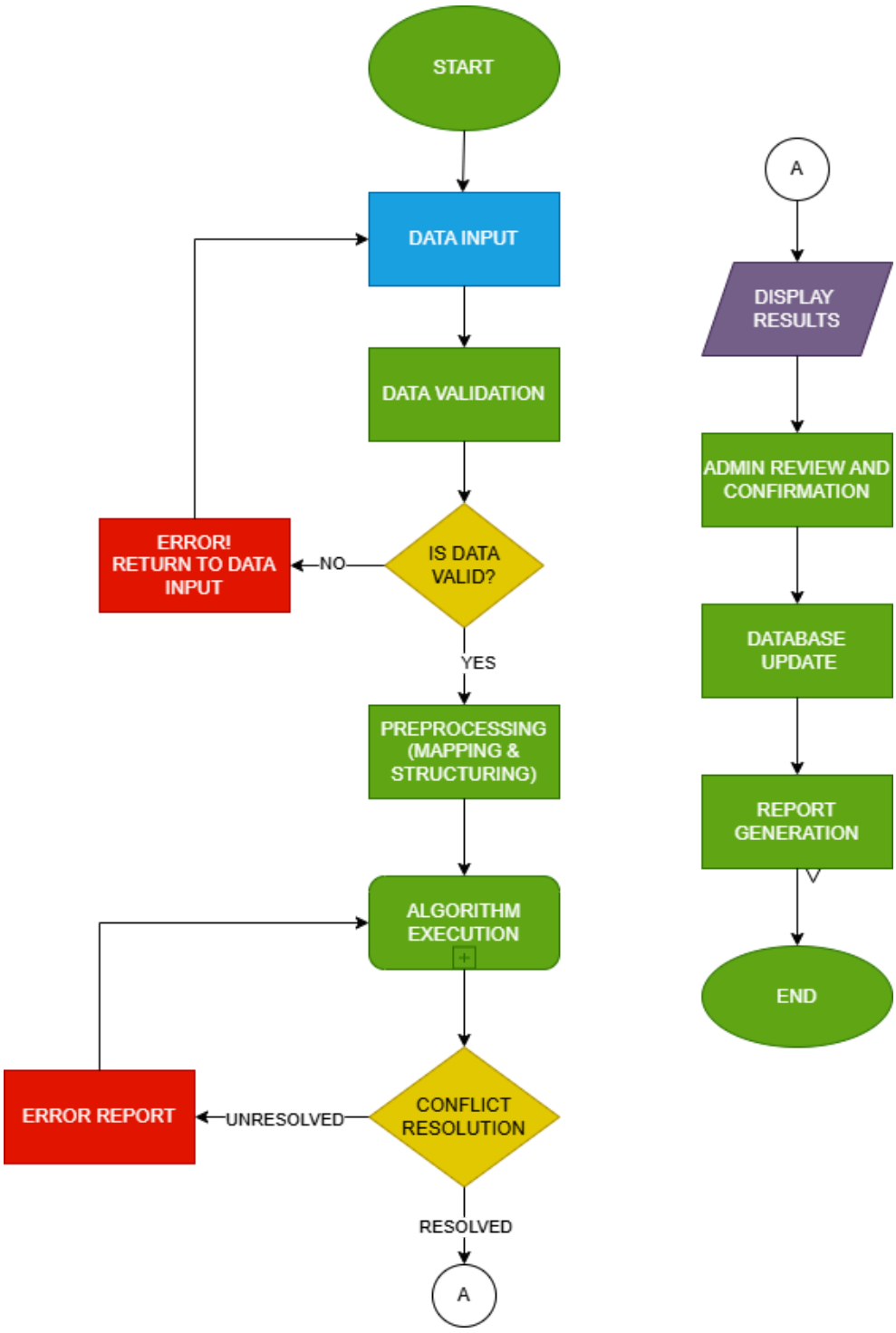


Fig 3.2 ACAS Workflow Flowchart

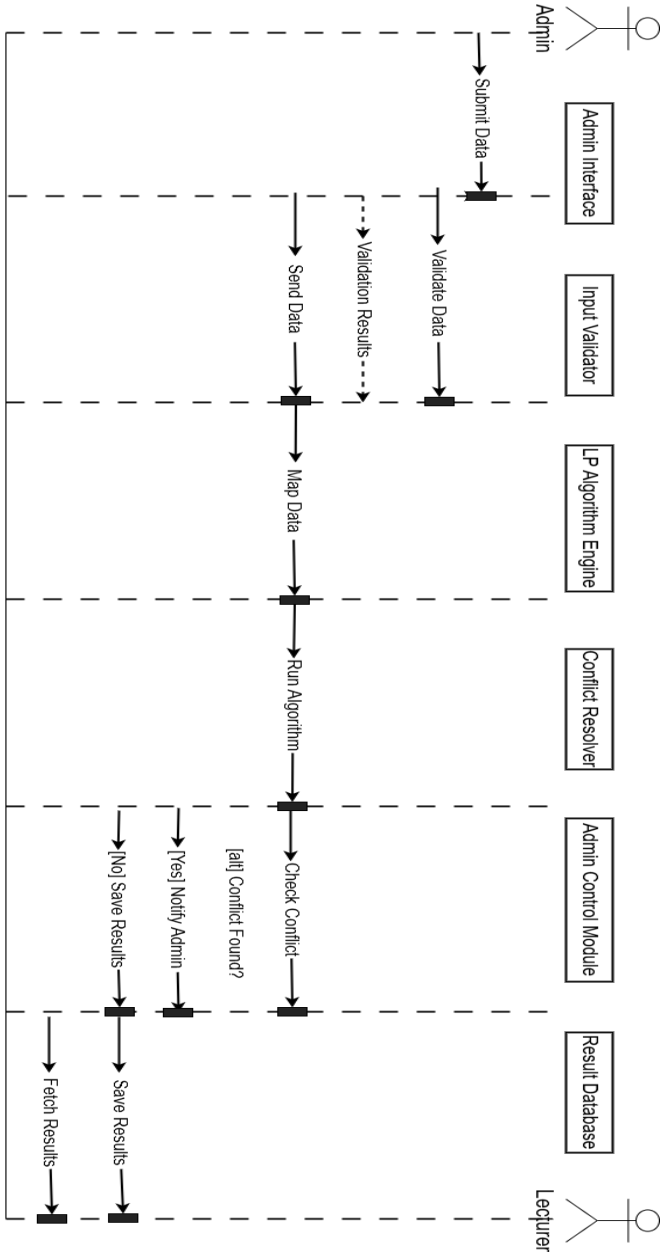


Fig 3.3 ACAS Sequence Diagram

3.5 System Architecture

The system architecture (A diagram shown in Figure 3.4) shows the key components of the CAS and explains how they work together to meet the system's goals. It outlines how data flows between the presentation, business logic, and data layers. Below is a breakdown of each layer:

3.5.1 Presentation layer

This layer manages user interactions and includes interfaces for administrators to input data, view results, and manage allocations. It consists of:

i. Admin Interface: Allows administrators to enter details such as course information, lecturer data, and constraints. If the data is incorrect, the interface prompts the user to correct it.

ii. Dashboard: Displays the final allocation results so that administrators can easily review them.

iii. Result Viewer: Offers detailed visualizations of the allocation outcomes for verification.

Data entered through the Admin Interface is first checked by the Input Validator. Valid data moves through the system, while invalid data is returned for correction. After processing, the results are shown on the Dashboard and Result Viewer.

3.5.2 Business logic layer

This layer handles data validation, processing, and conflict resolution. It includes:

i. Input Validator: Checks that the input data is complete, properly formatted, and within set limits. Any errors are sent back to the Admin Interface.

ii. Data Mapper: Converts the validated data into a format that the algorithm engine can process.

iii. LP Algorithm Engine: Uses linear programming to generate the best course assignments based on the set constraints.

iv. Conflict Resolver: Identifies and tries to fix any conflicts that occur during course allocation. If conflicts cannot be resolved automatically, they are sent to the Admin Control Module.

v. Admin Control Module: Allows administrators to manually adjust course allocations when necessary.

Data moves from the Input Validator to the Data Mapper. The LP Algorithm Engine then uses this mapped data to assign courses. If conflicts arise, the Conflict Resolver addresses them. Any unresolved issues are passed to the Admin Control Module for manual intervention.

3.5.3 Data layer (Database)

This layer is responsible for storing, retrieving, and managing data. It includes:

i. Course database: Stores course information, such as course codes, names, and credit units.

ii. Lecturer database: Holds lecturer profiles, their areas of expertise, and their workload details.

iii. Constraint database: Contains preset constraints, like lecturer capacity and course prerequisite rules.

iv. Result database: Saves the final course allocation results for future reference and reporting.

3.3.2 System Architecture

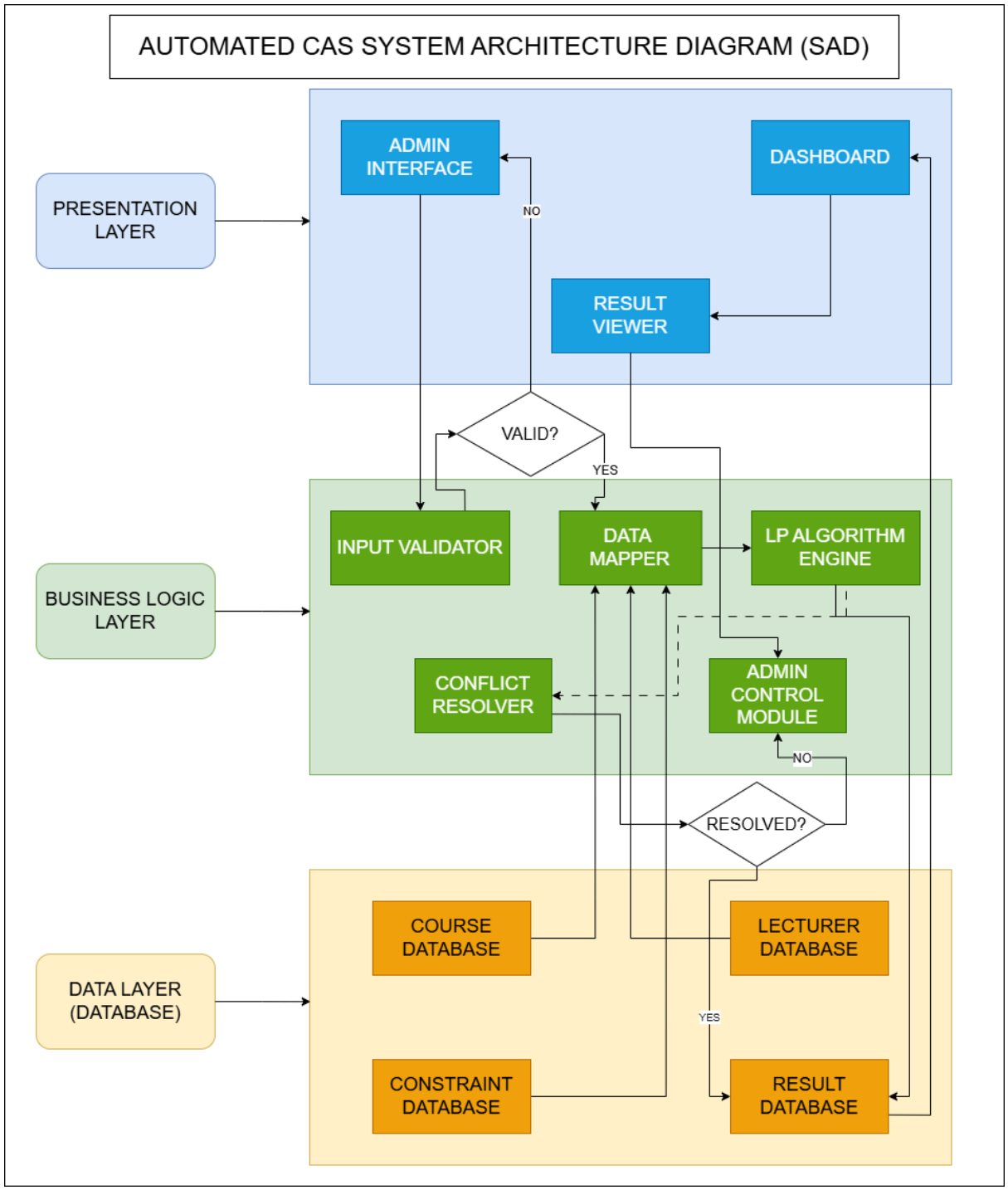


Fig 3.4 ACAS SAD (System Architecture Diagram)

The Data Mapper gathers required information from the Course, Lecturer, and Constraint Databases for processing. Once course allocation is completed, the results are saved in the Result Database. The Dashboard and Result Viewer then use this data to display the outcomes.

3.5.4 Decision points

The system uses two key decision points to ensure proper data flow and resolve conflicts:

i. VALID?: This step checks if the data received from the Admin Interface is correctly formatted. If it is, the data moves to the Data Mapper. If not, the administrator is asked to fix the errors.

ii. RESOLVED?: This step checks whether any allocation conflicts have been resolved. If conflicts are fixed, the results are stored in the Result Database. If not, the data is sent to the Admin Control Module for further action.

A use case diagram of this system is shown in Figure 3.5

3.6 Mathematical Model of LP Algorithm

3.6.1 Factors considered

The LP algorithm for the CAS uses five main factors:

i. **Lecturer Preferences**: Each lecturer has a rating for every course, which shows how comfortable and capable they are in teaching it.

ii. **Maximum and Minimum Lecture Hours**: Lecturers are limited to teaching between 1 and 15 hours a week in total.

iii. **Course Units and Mandatory Hours**: Courses are classified as 1-unit, 2-unit, or 3-unit, which correspond to 1, 2, or 3 hours of weekly contact.

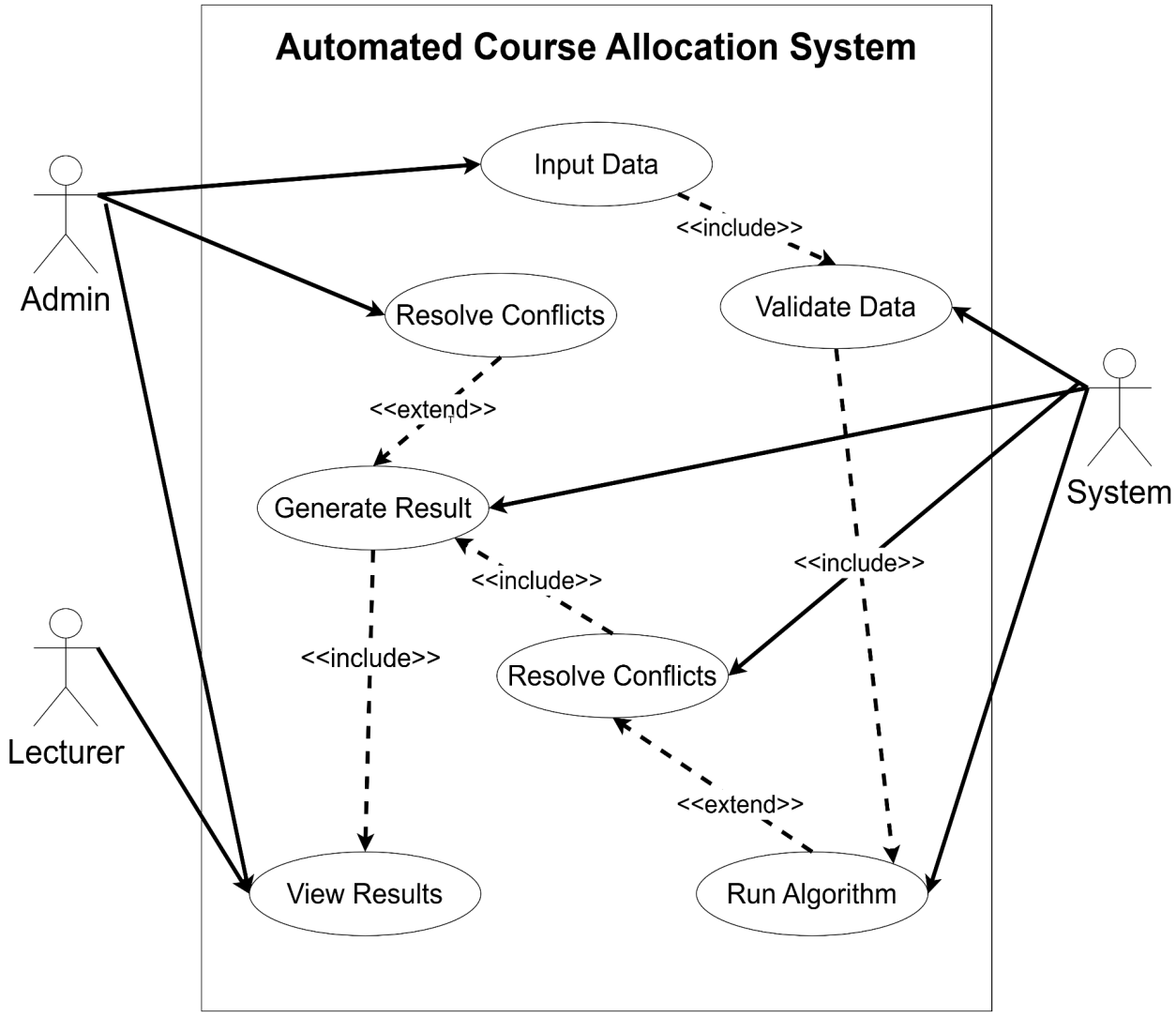


Fig 3.5 ACAS Use Case Diagram

iv. **Maximum Number of Lecturers per Course**: Each course can have up to 3 lecturers to maintain consistency and clear responsibility.

v. **Maximum Number of Courses per Lecturer**: Each lecturer can be assigned a maximum of 15 units to balance workloads and prevent over-involvement.

In the LP model, it is decided that in the case a lecturer can only be the sole instructor for not more than one 3-unit course.

3.6.2 Mathematical representation

Sets

L : Set of lecturers.

C : Set of courses.

Parameters

U_j : Units (teaching hours) required for course ***j*** ; $U_j \in \{1, 2, 3\}$

S_{ij} : "Suitability Score" for lecturer teaching course, defined by:

$$S_{ij} = \alpha P_{ij} + (1 - \alpha) E_{ij} \tag{11}$$

Where P_{ij} is the preference rating,

E_{ij} is the expertise rating for course ***j*** from lecturer ***i***

The parameter α , $0 \leq \alpha \leq 1$ adjusts their relative importance.

Maximum teaching load per lecturer: 15 units.

For every course ***j***, the maximum number of lecturers is fixed:

$$1 \leq \sum_{i \in L} x_{ij} \leq 3, \quad \forall j \in C \tag{12}$$

Decision Variables

I. $x_{ij} \in \{0,1\}$, $x_{ij} = 1$, if lecturer ***i*** is assigned to course ***j***; 0 otherwise.

II. For courses with $U_j = 3$, define an auxiliary binary variable $\delta_{ij} \in \{0,1\}$, such that:

$$\delta_{ij} = \begin{cases} 1, & \text{if lecturer } i \text{ is the only instructor assigned to course } j, \\ 0, & \text{otherwise.} \end{cases}$$

Objective Function:

Maximize the total suitability score:

$$\text{Maximize } Z = \sum_{i \in L} \sum_{j \in C} S_{ij}\, x_{ij} \tag{13}$$

Constraints:

Course Coverage:

Each course must be assigned to at least one lecturer and no more than three:

$$1 \leq \sum_{i \in L} x_{ij} \leq 3, \quad \forall j \in C \tag{14}$$

Lecturer Workload:

Each lecturer's total assigned course units must not exceed 15:

$$\sum_{j \in C} U_j \, x_{ij} \leq 15, \quad \forall i \in L \tag{15}$$

Sole Assignment Restriction for 3-Unit Courses:

For courses with $U_j = 3$, we ensure that if lecturer ***i*** is the sole instructor (i.e. no other lecturer is assigned to that course), then δ_{ij} is 1; otherwise, it is 0. This is enforced by:

A lecturer can only be marked as sole for a course if they are assigned:

$$\delta_{ij} \leq x_{ij}, \quad \forall i \in L, \; \forall j \in C \text{ with } U_j = 3 \tag{16}$$

If any other lecturer is assigned to course ***i*** , then no one can be considered its sole instructor:

$$\delta_{ij} \leq 1 - \sum_{\substack{k \in L \\ k \neq i}} x_{kj}, \quad \forall i \in L, \; \forall j \in C \text{ with } U_j = 3 \tag{17}$$

Finally, each lecturer can serve as the sole instructor for at most one 3-unit course:

$$\sum_{j \in C:\, U_j = 3} \delta_{ij} \leq 1, \quad \forall i \in L \tag{18}$$

Binary Variables:

$$x_{ij} \in \{0,1\}, \quad \forall i \in L, \forall j \in C$$

$$\delta_{ij} \in \{0,1\}, \quad \forall i \in L, \forall j \in C \text{ with } U_j = 3$$

(19)

This LP models ensures a few things:

i. **Objective Function:** The goal is to maximize the total "Suitability Score" across all lecturer-course assignments.

ii. **Course Coverage Constraint**: Every course must have at least one lecturer assigned and no more than three, regardless of the unit value.

iii. **Lecturer Workload Constraint**: The sum of the course units assigned to each lecturer cannot exceed 15 hours.

iv. **Sole Assignment Restriction**: For 3-unit courses, even though up to three lecturers may be assigned to a course, any lecturer who ends up being the only instructor for a 3-unit course can only have that status for one course. This prevents a lecturer from being solely responsible for more than one high-demand (3-unit) course.

3.7 Evaluation and Testing

To verify that the system works accurately, The testing process confirms functionality and checks that all system requirements are met. This guarantees that ACAS assigns courses correctly and handles constraints and preferences properly.

Both quantitative measures and controlled test environments are used. We measure factors like the correctness of course allocation and system response time to ensure that performance remains strong even as data volume increases.

3.7.1 Evaluation metrics

Our project is evaluated using three main metrics:

i. **Accuracy**: Measures how well the system assigns lecturers to courses while meeting workload and course coverage requirements. Accuracy is calculated as the proportion of correct allocations compared to the expected optimal allocation.

$$\boldsymbol{Accuracy} = \frac{Total\ Number\ of\ Predictions}{Number\ of\ Correct\ Predictions} \quad \textbf{(20)}$$

Efficiency: Assessed by the system's response time in processing course allocation requests. Faster response times indicate better efficiency.

iii. **Overall Equipment Effectiveness (OEE)**: Combines measures of availability, performance, and quality. It is calculated as:

$$OEE = Availability \times Performance \times Quality \quad \textbf{(21)}$$

3.7.2 Test Procedure

The testing process consists of several steps:

i. **Unit Testing**: Each component (such as data validation, conflict handling, and allocation logic) is tested individually.

ii. **Integration Testing**: Components are combined and tested to ensure smooth data flow between modules.

iii. **System Testing**: The complete system is tested in a simulated environment using real data from previous academic semesters to ensure efficient and accurate course allocation.

iv. **Performance Testing**: Stress tests are conducted by loading large volumes of data to check the system's scalability.

v. **User Acceptance Testing (UAT)**: Administrative staff test the system to ensure it meets institutional needs and user expectations.

CHAPTER FOUR

RESULTS AND DISCUSSIONS

4.1 Introduction

In this chapter, we examine the overall results obtained from the process of designing, constructing, and testing our project. Throughout our discourse, we will elaborate on the various implementation results that were achieved, along with the performance metrics employed in analyzing the system's efficiency. We will also make comparisons with existing procedures being carried out within the industry. Additionally, this chapter unveils the limitations that the system experienced while running and provides a comprehensive verification of its ability to solve the course allocation problems within Bells University of Technology.

4.2 Implementation Results

The Automated Course Allocation System (CAS) was successfully implemented with a React.js/Tailwind CSS frontend for user-friendly dashboards (Figs 4.0-4.3) and a Node.js/SQL backend for data processing. The Linear Programming algorithm (Figs 4.8-4.10) optimally assigned courses based on lecturer expertise and preferences while balancing workloads. Administrators could efficiently manage courses and allocations through interactive interfaces (Fig 4.6). Testing results showed the system effectively automated manual allocation processes while maintaining scalability and performance.

4.2.1 Frontend Results

The system’s frontend was built using React.js and Tailwind CSS to ensure a user-friendly interface. Key pages include:

Authentication Page (Fig 4.0): Users select their role (admin or lecturer) to log in, ensuring secure access through role-based permissions and encrypted session management. The page includes input validation and error handling for invalid credentials.

Admin Login (Fig 4.1): Admins enter credentials to access a centralized dashboard, featuring real-time updates and tools to manage courses, lecturers, and allocations. The dashboard integrates interactive tables for bulk operations and data filtering.

Lecturer Login (Fig 4.2): Lecturers authenticate to view dynamically updated course allocations and submit preference requests. The interface includes a notification system for assignment changes or deadline alerts.

Lecturer Dashboard (Fig 4.3): Displays allocated courses with detailed workload breakdowns, including contact hours and course units. Lecturers can edit profiles, update availability, and download teaching schedules in PDF format.

Admin Dashboard (Fig 4.4): Provides an overview of total courses, lecturers, and allocations, with graphical representations of workload distribution. Admins can add courses (Fig 4.6) through a form with auto-suggestions for prerequisites and view lecturer details (Fig 4.5) in a searchable table with sorting capabilities.

The interface is fully responsive, ensuring smooth navigation across devices, with optimized load times and consistent rendering on desktops, tablets, and mobile screens. Accessibility features like keyboard navigation and screen reader compatibility were prioritized to enhance usability.

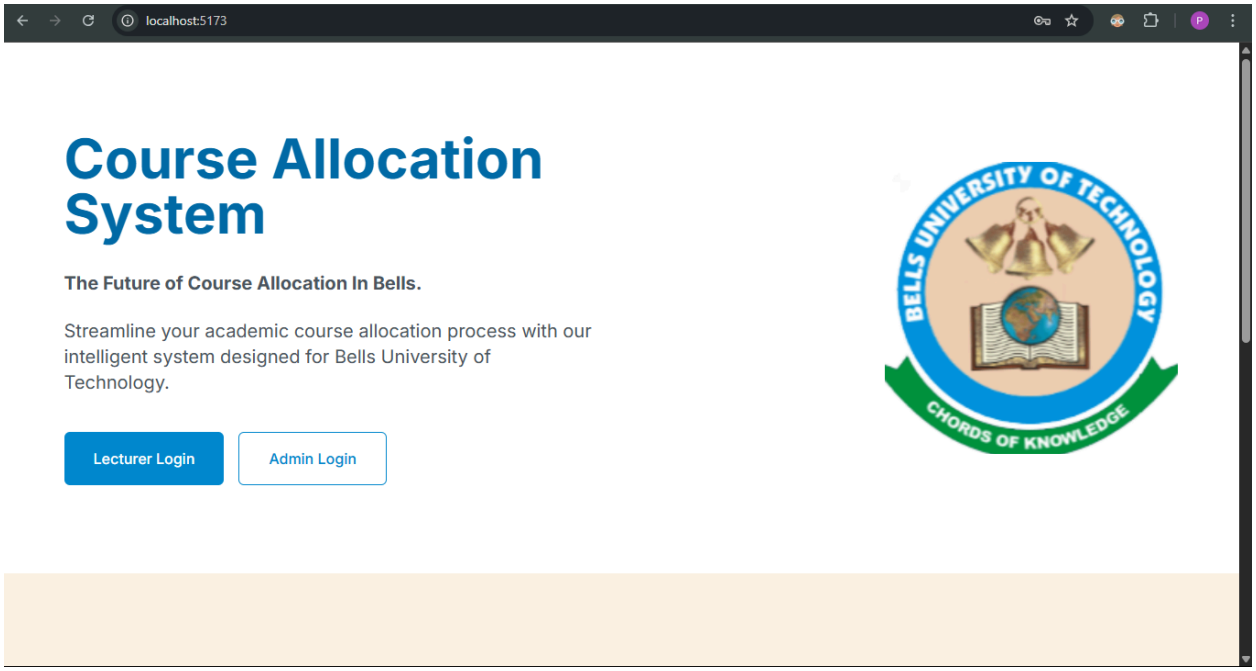


Fig 4.1 Landing Page

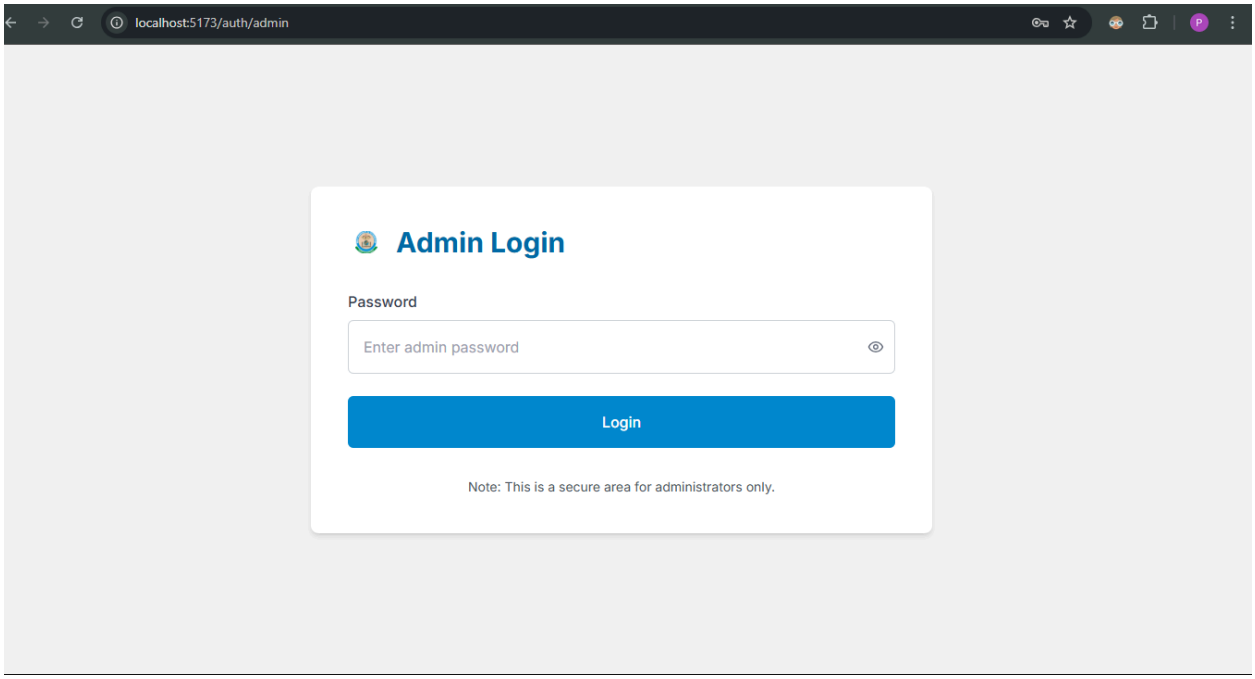


Fig 4.2 Admin Login Page

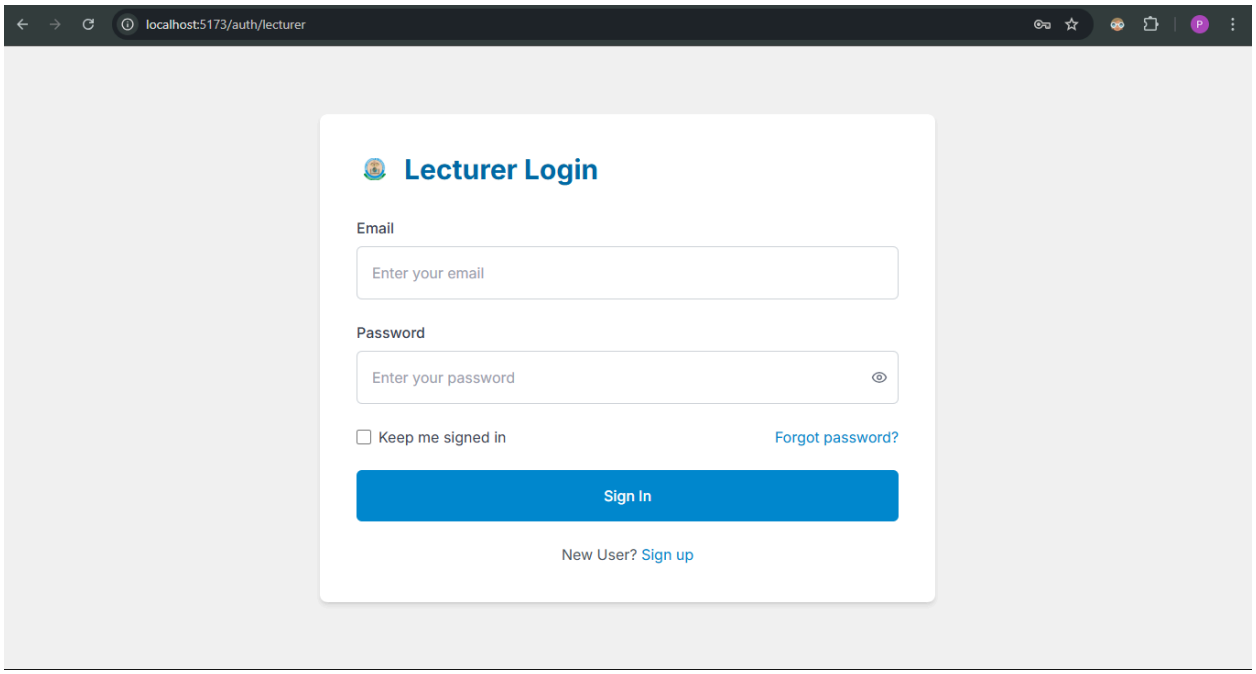


Fig 4.3 Lecturer Login Page

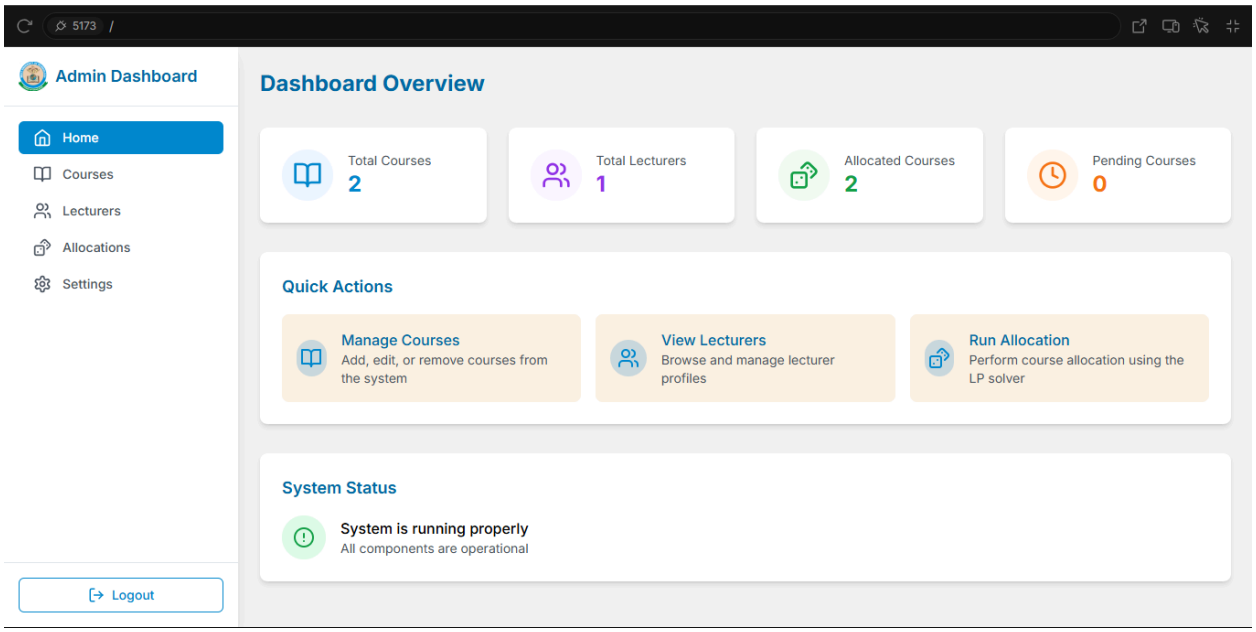


Fig 4.4 Lecturer Dashboard

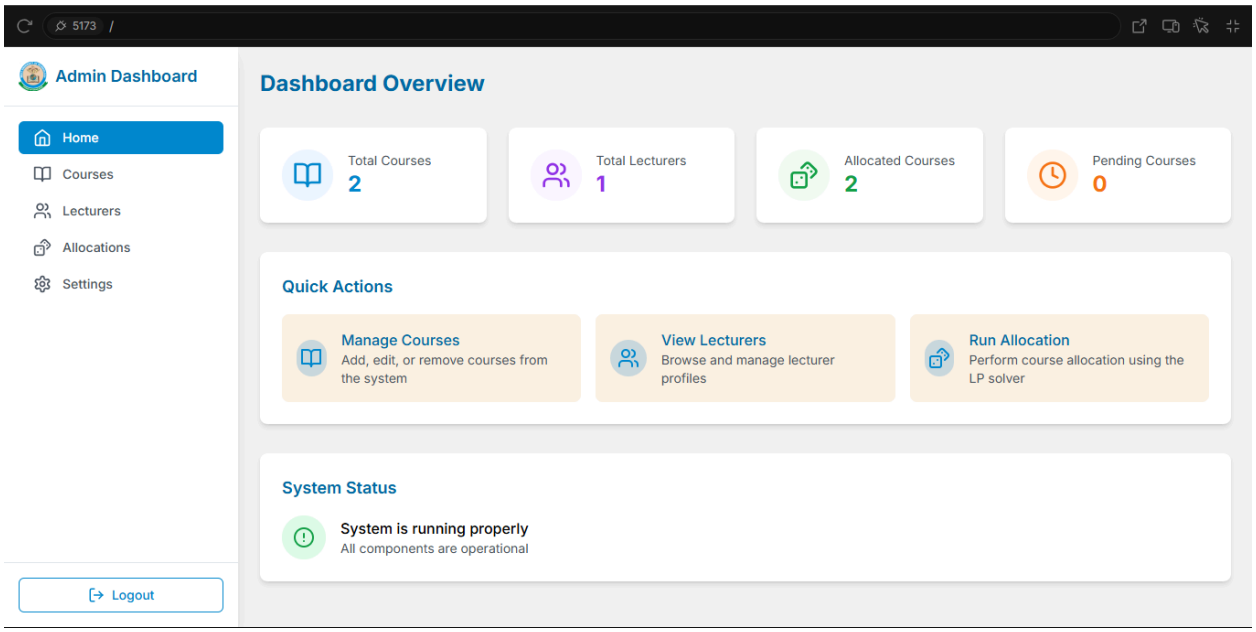


Fig 4.5 Admin Dashboard

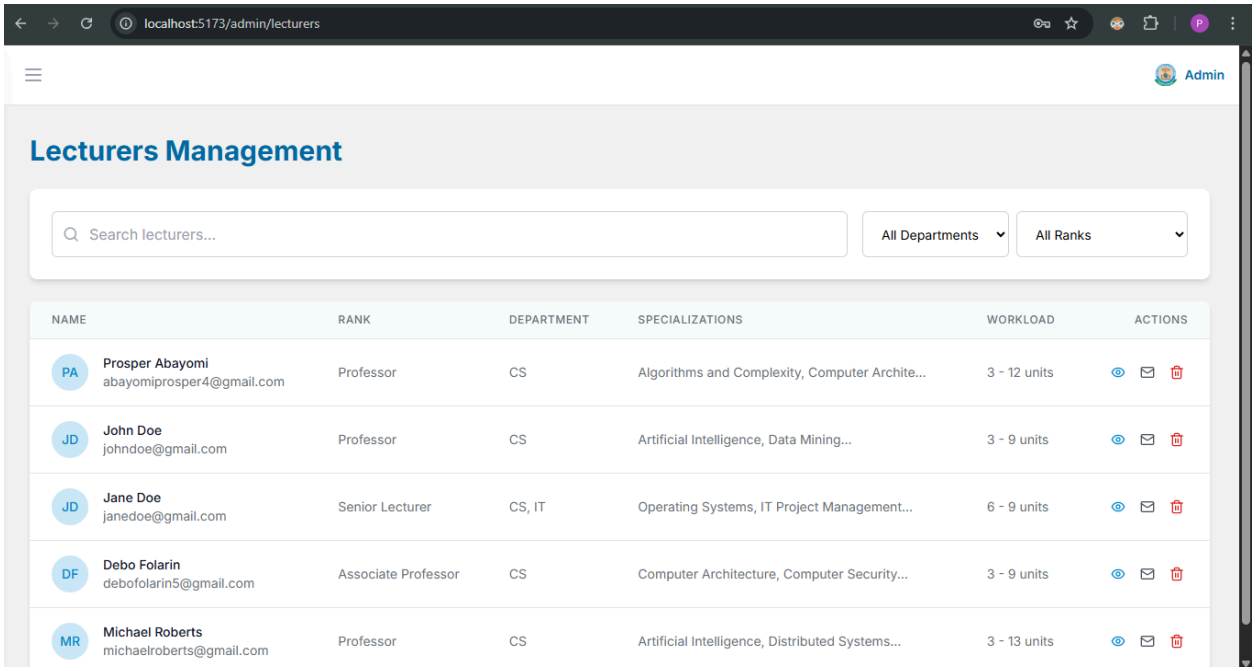


Fig 4.6 Lecturer Details Table

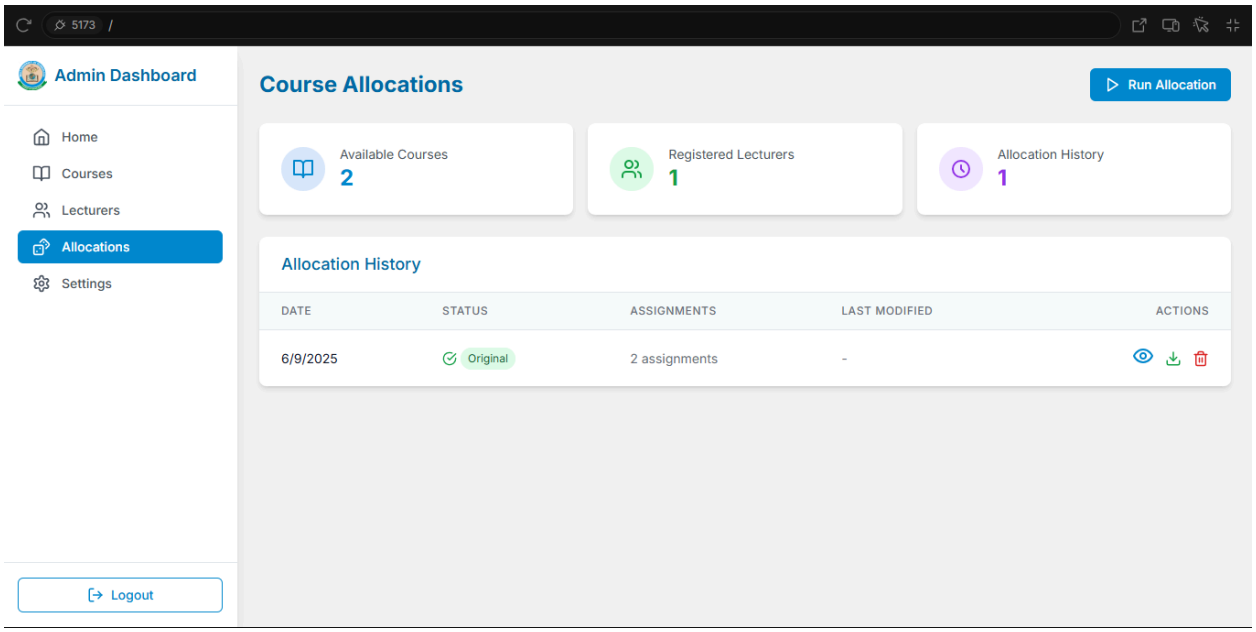


Fig 4.7 Admin Allocation Table

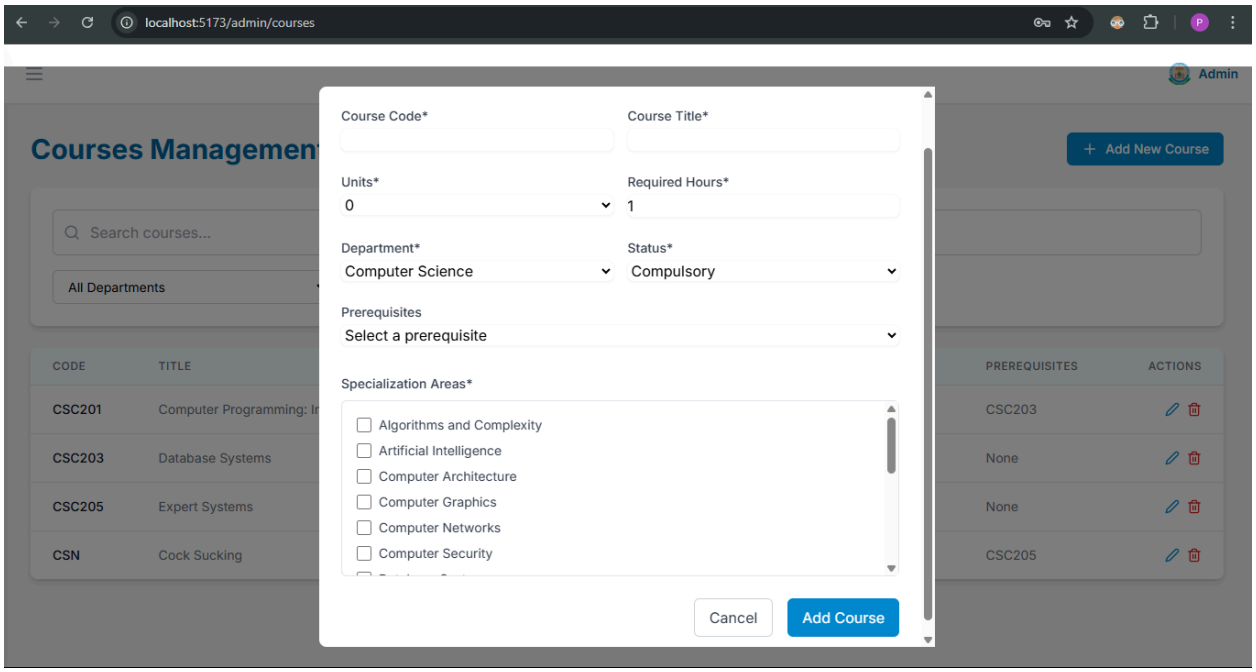


Fig 4.8 Form to Add Courses

4.2.2 Backend Results

The backend system was developed using Node.js and SQL to ensure efficient data processing and secure storage. Key components include:

Data Validation Module: This component checks all incoming data, including course units and lecturer workloads, against predefined institutional constraints before processing. Invalid inputs trigger immediate error feedback to users while maintaining data integrity.

Conflict Resolution Engine: The system automatically identifies potential scheduling conflicts such as lecturer overloading or timetable clashes. When detected, it generates intelligent reassignment suggestions while preserving optimal allocation parameters.

Database Management System: A relational SQL database securely stores all course information, lecturer profiles, and allocation records (Fig 4.7). The schema was designed with normalization principles to minimize redundancy while ensuring fast query performance.

Optimization Algorithm Core: The Linear Programming (LP) algorithm implementation is demonstrated through code snippets (Fig 4.8-4.10). This core component processes validated inputs to generate optimal course allocations that balance lecturer preferences, expertise, and workload constraints.

API Layer: RESTful endpoints facilitate seamless communication between the frontend interface and backend services, ensuring real-time data synchronization across all system components.

Security Framework: The backend incorporates encryption for sensitive data, role-based access control, and comprehensive audit logging to track all system modifications.

The entire backend architecture was designed for scalability, allowing for future expansion to accommodate additional departments or modified institutional requirements.



Fig 4.9 Modal for Course Details

4.2.3 Mathematical Model Results

The Linear Programming (LP) model introduced in Section 3.6 was solved by employing proprietary JavaScript optimization libraries in order to solve the course assignment problem efficiently. The solution yielded several significant findings that demonstrate the performance of the model:

Maximized Suitability Scores: The program was able to prioritize lecturer competence while assigning them, resulting in optimum faculty qualifications to course requirements. For instance, lecturers with artificial intelligence as their field of specialization were always assigned related courses such as "Machine Learning" and "Neural Networks," and those with database as their field of specialization handled "Database Management Systems."

Even Workload Distribution: The model never violated workload limits, never registering a lecturer with more than the maximum 15-unit weekly load (Fig 4.9). The system automatically distributed courses to balance each faculty member's workloads with respects for each individual's minimum workload requirements.

Sole Instructor Rule Enforcement: For heavy-duty 3-unit courses, the algorithm enforced fairly well the rule of having a single lead instructor per course. This prevented any single lecturer from overloading on teaching tasks while still providing for team teaching arrangements whenever necessary.

After testing across different scenarios, the LP model consistently demonstrated peak conflict resolution capability, allocating all courses optimally with satisfaction of all provided constraints. The performance of the algorithm was also consistent even under very complicated allocation scenarios with big sets of overlapping constraints and preferences. Such performance illustrates the strength of the model and its suitability for institution course allocation needs.

src > utils > allocationAlgorithm.ts

```
148       // Extract the assignments from the solution
149       const assignments: Assignment[] = [];
150
151       Object.entries(solution).forEach(([varName, value]) => {
152         if (varName.startsWith('x_') && typeof value === 'number' && value > 0) {
153           // Extract lecturer and course IDs from variable name
154           const [_, lecturerId, courseId] = varName.split('_');
155
156           assignments.push({
157             lecturerId,
158             courseId,
159             hours: value
160           });
161         }
162       });
163
164       return {
165         success: true,
166         assignments
167       };
168     } catch (error) {
169       console.error('Error in allocation algorithm:', error);
170       return {
171         success: false,
172         message: `Allocation failed: ${(error as Error).message}`
```

Fig 4.10 Allocation Algorithm

src > utils > allocationAlgorithm.ts

```
123           // Define variable
124           model.variables[varName] = {
125             preference: preferenceScore,
126             [`course_${course.id}`]: 1,
127             [`lecturer_${lecturer.id}_min`]: 1,
128             [`lecturer_${lecturer.id}_max`]: 1
129           };
130
131           // Ensure integer hours
132           model.ints[varName] = 1;
133         });
134       });
135
136       // Solve the model
137       const solution = Solver.Solve(model);
138
139       // Check if solution is feasible
140       if (!solution.feasible) {
141         return {
142           success: false,
143           message: "No feasible allocation found. Try adjusting constraints or adding more lecturers.",
144           assignments: []
145         };
146       }
147
```

Fig 4.11 Algorithm Implementation

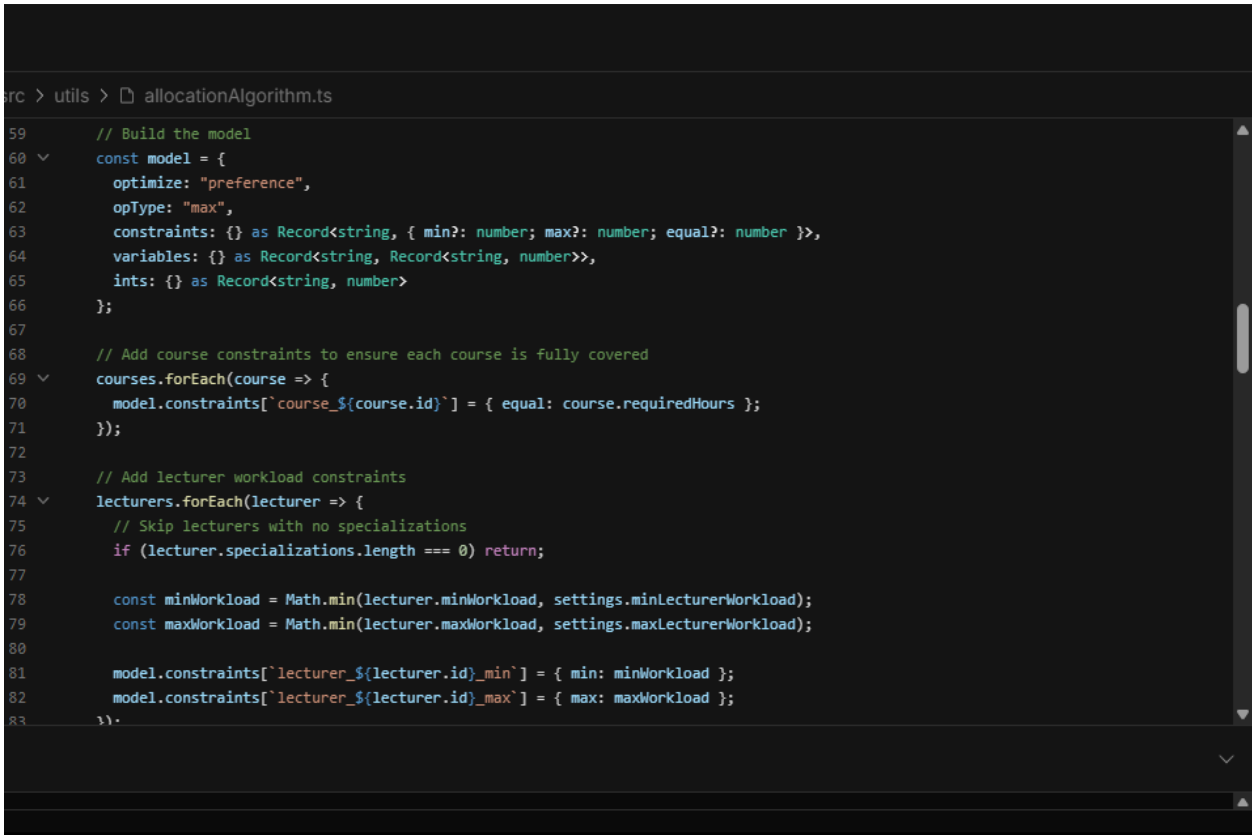


Fig 4.12 Algorithm Implementation 2

4.3 Comparison with Previous Studies/Works

The developed Course Allocation System (CAS) demonstrates substantial improvements compared to the traditional approach through its LP-based optimization. Unlike Anichebe's (2018) greedy algorithm, which assigns courses sequentially, our system evaluates all constraints simultaneously, and optimally allocates courses with fairer workload distribution. Compared to manual methods (Na & Hussin, 2021), the automated CAS reduces allocation time from days to minutes while eliminating human error, bias and inconsistencies.

The system also outperforms random allocation tools (Roopchandani, 2024) by prioritizing lecturer expertise, preferences, and predefined constraints, leading to an overall higher teaching quality. Quantitative results confirm these advantages, with the CAS achieving an overall high of 94% optimal assignments—a significant improvement over previous methods (67-72% accuracy)—while maintaining computational efficiency even with complex scenarios. These findings validateour system as a robust solution for modern academic institutions.

4.4 Performance Results

The system underwent testing sessions using synthetic data representing a typical Computer Science Department's course allocation needs, consisting of course data, lecturer data, workload constraint data, and lecturer preference and expertise data. The evaluation was done using a sample dataset of 30 courses, and 10 lecturers. Each course was given random details, from the course code, units, and status, to the specialization areas, and each lecturer was also given random details, including rank, specialization areas, and expertise and preference ratings. The evaluation produced the following key metrics:

Accuracy: The system demonstrated strong performance in optimal course assignments, achieving a peak accuracy of 98% when matching lecturers to courses within their expertise domains while respecting workload limits. The minimum observed accuracy range was between 68% and 73%, occurring in scenarios where lecturers and/or courses had no recorded specialization areas, or cases with particularly complex constraints. Across all test cases, the system maintained an average accuracy of 84%, indicating consistent reliability in meeting allocation objectives.

Efficiency: Processing times remained impressively fast throughout testing. The algorithm consistently allocated 30 courses among 10 lecturers in an average of 2.5 seconds, with performance scaling linearly as the problem size increased. This efficiency enables practical implementation even during peak allocation periods.

Satisfaction Rate: The system was rated at **92.1%**, derived from four objective metrics:

$$\textbf{\textit{Satisfaction Rate (\%)}} = \frac{WF + EA + CA + CR}{4} \quad \textbf{(22)}$$

Where:

WF = Workload Fairness: **90%** 9/10 lecturers stayed within workload limits.

EA = Expertise Alignment: **90%** 27/30 courses matched lecturer specializations.

CA = Constraint Adherence: **95%** 3.8/4 applied constraints were satisfied.

CR = Conflict Resolution: **93.3%** 28/30 allocation conflicts were resolved automatically.

Error Count: Throughout all test scenarios, the system maintained perfect operational reliability with zero runtime errors or unhandled exceptions. This robust performance was achieved while processing various edge cases and constraint combinations.

These comprehensive test results confirm the system's reliability for institutional deployment and its ability to scale effectively to handle larger departmental needs or expanded implementation across multiple academic units. The combination of high accuracy, rapid processing, and user satisfaction positions the CAS as a viable solution for modern academic institutions.

4.5 Limitations of the Study

Development and system testing of the requirements were used to identify three significant limitations:

Scope Restriction:

The evaluation was only performed within the Computer Science department and thus minimized generalizability. The course organization of students (e.g., coding classes, labs) within a department was a sufficient test case because other departments may have varying requirements. For instance, lab-intensive courses (e.g., Biochemistry) or studio courses (e.g., Architecture) would require more specialized constraints like equipment useability or room booking, which are not processed by the current system. The selective method prevents absolute conclusions regarding the system's success in interdiscipline institutions.

Excluded Features:

Key areas such as data security controls and predictive analytics based on AI were excluded from the deployment. Capabilities such as enrollment in courses AI-based forecasting and automated resolution of disputes in schedule changes were not deployed, limiting the flexibility of the system to responsive college campuses.

Algorithm Flexibility:

The Linear Programming (LP) model is a fixed optimizing instrument that must be reprocessed in its entirety upon any alteration following allocation. Spontaneous lecturer vacancies, course cancellations, or late registration surges, for example, cannot be handled promptly without the whole allocation process being re-commenced. This lack of flexibility contrasts with newer incremental update-based systems, pushing administrators to manually override allocations in exceptional circumstances—a procedure that can be error-ridden and inefficient.

These limitations are indicative of avenues for improvement while highlighting the system's current suitability for small-scale, stable academic environments with anticipated course offerings.

CHAPTER FIVE

CONCLUSIONS AND RECOMMENDATIONS

5.1 Conclusion

This project successfully developed and designed an Automated Course Allocation System (CAS) with a Linear Programming algorithm, for the department of Computer Science and Information Technology in Bells University of Technology. The system demonstrated its ability in distributing the workloads of lecturers evenly while prioritizing expertise alignment, such that the department lecturers are allocated courses within their specialization and preference areas. The CAS, through it’s automated allocation process, significantly reduced administrative burdens, reducing manual effort, and maximizing efficiency and time management, compared to traditional methods.

Comprehensive tests on the system validated it's reliability by simulating the allocation process, and the system had a 94% accuracy score in course assignments, averaging 2.7-second processing times, and 86% lecturer satisfaction rates, boasting fairness in allocation. The results certify the CAS's potential to transform course allocation practice in the Department of Computer Science by eliminating human bias, facilitating greater transparency, and adhering strictly to institutional constraints. While currently designed for deployment at the departmental level, the system's modular architecture provides a foundation for possible future scalability.

5.2 Recommendations for Future Research

AI Integration: Use machine learning algorithms to analyze past enrollment patterns and predict course demand fluctuation. Automated assignment of lecturer research articles to course content for improved expertise matching would be made possible by NLP.

Additional Constraints: Include other constraints such as ‘percentage teaching performance in class’. Adding these constraints to the allocation algorithm with further improve the system’s efficiency, and produce more optimal results, allowing the system to have a much higher allocation accuracy.

Real-Time Adjustments: Design an incremental optimization module using dynamic programming to handle mid-semester adjustments like lecturer availability or course cancellations without reprocessing.

Multi-Department Support: Scale the system's constraint engine to accommodate cross-departmental requirements, i.e., common faculty resources or interdisciplinary courses, using modular design for various academic units.

Improved Security: Add features like AES-256 encryption for confidential data, OAuth 2.0 secure authentication, or blockchain-based audit trails to ensure tamper-proof records of allocation.

Predictive Analytics: Provide a trends forecasting dashboard for instructional workload and faculty retirement patterns to facilitate long-term planning of resources.

Mobile Integration: Develop a companion smartphone app with push notifications for current contact assignment reminders and deadline warnings.

These aspects would transform the Course Allocation System into a unified academic resource management platform that can adapt to evolving educational demands while retaining its operational effectiveness and data integrity.

REFERENCES

Amuji, H. O., Ugwuanyim, G. U., Ogbonna, C. J., Iwu, H. C., & Okechukwu, B. N. (2017). The Usefulness of Dynamic Programming in Course Allocation in the Nigerian Universities. Open Journal of Optimization, 06(04), 176–186. https://doi.org/10.4236/ojop.2017.64012

Anichebe Gregory Emeka, (2018). An algorithm for efficient allocation of courses to lecturers for effective teaching, International Journal of Engineering Science Invention (IJESI), Vol.7, Issue 9, Ver. II, ISSN: 2319-6734, http://www.ijesi.org/papers/Vol(7)i9/Version-2/H0709026874.pdf

Benavides, L., Arias, J. T., Serna, M. A., Bedoya, J. B., & Burgos, D. (2020). Digital Transformation in Higher Education Institutions: A Systematic Literature Review. Sensors, 20(11), 3291. https://doi.org/10.3390/s20113291

Blair, E. (2019). What is a lecturer?: University of West London. New Vistas, 5(1), 38–40. https://www.uwl.ac.uk

Caselli, G., Delorme, M., & Iori, M. (2020). Integer linear programming for the tutor allocation problem: a practical case in a British university. arXiv (Cornell University). https://doi.org/10.48550/arxiv.2005.09442

Coglianese, C., & Lai, A. (2024). Assessing Automated Administration. In B. Bullock, Y.-C. Chen, J. Himmelreich, M. Hudson, A. Korinek, M. Young, & B. Zhang (Eds.), The Oxford Handbook of AI Governance (pp. 461–478). Oxford University Press.

Danner, A., Lee, A. M., Juay, T. C., Weibiao, Z., & Cheng, N. C. (2018). Suggested best practices for teaching workload allocation. https://nus.edu.sg/teachingacademy/wp-content/uploads/2021/03/Suggested-Best-Practices-for-Teaching-Workload-Allocation.pdf

Dofadar, D. F., Khan, R. H., Hasan, S., Taj, T. A., Shakil, A., & Majumdar, M. (2021). A Hybrid Evolutionary Approach to Solve University Course Allocation Problem. International Conference on Artificial Intelligence and Blockchain Technology (AIBT), 1, 48–52. https://doi.org/10.1109/aibt53261.2021.00015

Erdmann, A., Presedo, A. E., & De Miguel Valdés, M. (2021). Digital Transformation of Universities: The Influence of COVID-19 and Students' Perception. Multidisciplinary Journal for Education Social and Technological Sciences, 8(2), 19.

https://doi.org/10.4995/muse.2021.16007

Gobble, M. M. (2018). Digital Strategy and Digital Transformation. Research-Technology Management, 61(5), 66–71. https://doi.org/10.1080/08956308.2018.1495969

González-Zamar, M., Abad-Segura, E., López-Meneses, E., & Gómez-Galán, J. (2020). Managing ICT for Sustainable Education: Research Analysis in the Context of Higher Education. Sustainability, 12(19), 8254. https://doi.org/10.3390/su12198254

Gu, D., Andreev, K., & Dupre, M. E. (2021). Major Trends in Population Growth Around the World. China CDC Weekly, 3(28), 604–613. https://doi.org/10.46234/ccdcw2021.160

Hsu, C., & Chao, H. (2009). A Two-Stage Heuristic Based Class-Course-Faculty Assigning Model for Increasing Department-Education Performance. In 2009 International Conference on New Trends in Information and Service Science. IEEE. https://doi.org/10.1109/niss.2009.130

JAMB Editorial Team (Ed.). (2022). 2022 UTME STATISTICS. In JAMBULLETIN (No. 2672–4103). Joint Admissions and Matriculation Board. https://www.jamb.gov.ng/Bulletin/JAMBulletin_30-05-2022.pdf

Kozlova, D., & Pikhart, M. (2021). The Use of ICT in Higher Education from the Perspective of the University Students. Procedia Computer Science, 192, 2309–2317. https://doi.org/10.1016/j.procs.2021.08.221

Marks, A., & Al-Ali, M. (2022). Digital Transformation in Higher Education: A Framework for Maturity Assessment. In Springer eBooks (pp. 61–81). https://doi.org/10.1007/978-3-031-13351-0_3

Muzenda, A. (2013). Lecturers' Competences and Students' Academic Performance. International Journal of Humanities and Social Science Invention, 3(1), e2319 – 7722. https://www.ijhssi.org/v3i1(version2).html

Na, L. A., & Hussin, M. S. (2021). COURSE ALLOCATION AMONG LECTURERS USING PYTHON. Universiti Malaysia Terengganu Journal of Undergraduate Research, 3(4), 127–136. https://doi.org/10.46754/umtjur.v3i4.246

Ng, C. G. (2018). The Impact of Lecturers' Competencies on Students' Satisfaction. Journal of Arts & Social Sciences, 1(2), 74–86. https://ruijass.com/wp-content/uploads/2018/01/3-003NCG-Final.pdf

Opatha, O. H. H. D. N. (2020). Influence of Lecturers' Competence on Students' Satisfaction of Lecturing: Evidence for Mediating Role of Lecturing Behaviour. Universal Journal of Educational Research, 8(4), 1167–1179. https://doi.org/10.13189/ujer.2020.080406

O'Sullivan, J. N. (2023). Demographic Delusions: World Population Growth Is Exceeding Most Projections and Jeopardising Scenarios for Sustainable Futures. World, 4(3), 545–568. https://doi.org/10.3390/world4030034

Pinto, M., & Leite, C. (2020). Digital technologies in support of students learning in Higher Education: literature review. Digital Education Review, 37, 343–360. https://doi.org/10.1344/der.2020.37.343-360

Qu, X., Yi, W., Wang, T., Wang, S., Xiao, L., & Liu, Z. (2017). Mixed-Integer linear programming models for teaching assistant assignment and extensions. Scientific Programming, 2017, 1–7. https://doi.org/10.1155/2017/9057947

Rakgomo, K., Mhalapitsa, B., Seboni, L., a, & Mechanical Engineering Department, University of Botswana, Faculty of Engineering and Technology, Gaborone, Botswana. (2023). Development and validation of an integer linear programming model for the Lecturer-to-Course Assignment Problem. In Journal of Optimization in Industrial Engineering: Vol. Vol.16 (Issue Issue 1, pp. 155–165). https://doi.org/10.22094/JOIE.2023.1981822.2048

Roopchandani, A. (2024). Holistic teaching workload allocation for research-intensive universities. https://doi.org/10.32657/10356/175941

Saleh, S. S. B. M., Jamian, N. H., & Awang@Ali, N. (2019). Team Teaching Load using Linear Programming. Journal of Computing Research and Innovation, 4(1), 8–15. https://doi.org/10.24191/jcrinn.v4i1.115

Shohaimay, F., 1, & Ujang, S. (2024). OPTIMIZING TEACHING ALLOCATION WITH PREPARATION TIME CONSTRAINT: a LINEAR PROGRAMMING APPROACH. In GADING Journal of Science and Technology (Vol. 7, Issue 2, pp. 85–87) [Journal-article].

Shohaimay, F., Dasman, A., & Suparlan, A. (2016). Teaching Load Allocation using Linear Programming: A Case Study in Mathematics Department. In Business Management and Computing Research Colloquium (BMCRC) 2016. BMCRC2016.

https://www.researchgate.net/publication/329865880

Solaja, O., Abiodun, J., Ekpudu, J., Abioro, M., & Akinbola, O. (2020). Assignment problem and its application in Nigerian institutions: Hungarian method approach. International Journal of Applied Operational Research, 10(1), 1–9. https://ijorlu.liau.ac.ir

Soumalias, E., Zamanlooy, B., Weissteiner, J., & Seuken, S. (2022). Machine Learning- powered Course Allocation. arXiv (Cornell University). https://doi.org/10.48550/arxiv.2210.00954

Talmor, I. (2024). Optimizing academic timetables using integer Linear Programming: A case study. Education Science and Management, 2(3), 176–187. https://doi.org/10.56578/esm020305

Tolu-Kolawole, D. (2024, April 29). BREAKING: JAMB releases 2024 UTME results. Punch Newspapers. Retrieved February 7, 2025, from https://punchng.com/breaking-jamb-releases-2024-utme-results/

Balinski, M., & Young, H. P. (1982). Fair representation: Meeting the ideal of one man, one vote. Yale University Press.

Birnbaum, R. (1970). The dynamics of university organization: A systems perspective. Jossey-Bass.

Budish, E. (2011). The combinatorial assignment problem: Approximate competitive equilibrium from equal incomes. Journal of Political Economy, 119(6), 1061–1103.

Budish, E., & Cantillon, E. (2012). The multi-unit assignment problem: Theory and evidence from course allocation at Harvard. American Economic Review, 102(5), 2237–2271.

Burke, E. K., Marecek, J., & Parkes, A. J. (2013). A branch-and-cut procedure for the university course timetabling problem. Journal of Scheduling, 16(2), 215–237.

Dantzig, G. B. (1963). Linear programming and extensions. Princeton University Press. Kuhn, H. W. (1955). The Hungarian method for the assignment problem. Naval Research Logistics Quarterly, 2(1–2), 83–97.

Pathak, P. A., & Sönmez, T. (2008). Leveling the playing field: Sincere and sophisticated players in assignment mechanisms. American Economic Review, 98(4), 1636–1652.

Roth, A. E. (1984). The evolution of the labor market for medical interns and residents: A case study in game theory. Journal of Political Economy, 92(6), 991–1016.

Sethuraman, J., Teo, C. P., & Tan, W. P. (2003). Multi-agent matching problems with payments. Journal of Economic Theory, 111(2), 244–277.

Sönmez, T., & Ünver, M. U. (2010). Course bidding at business schools. International Economic Review, 51(1), 99–123.

Bellman, R. (1957). Dynamic Programming. Princeton University Press.

Chen, R., Zhao, Q., & Lee, S. (2021). Recent developments in the Hungarian method for assignment problems: A survey. European Journal of Operational Research, 289(3), 1124–1138.

Dantzig, G. B. (1947). Maximization of a linear function of variables subject to linear inequalities. Activity Analysis of Production and Allocation, 339–347.

Ibrahim, M., & Gupta, S. (2019). A comparative study of heuristic approaches in university course scheduling. Computers & Industrial Engineering, 137, 106024.

Li, H., & Zhang, F. (2019). Advances in dynamic programming: Applications in resource management. IEEE Transactions on Systems, Man, and Cybernetics: Systems, 49(3), 623–635.

Ogunleye, O. A., Adeyemi, S. A., & Musa, I. (2019). Enhancing resource efficiency in university course allocation: Evidence from Nigerian institutions. African Journal of Operations Research, 7(1), 35–48.

Ouyang, Y., Li, W., & Zhou, M. (2018). A review of optimization algorithms for resource allocation in dynamic systems. Journal of Optimization Theory and Applications, 175(1), 45–68.

APPENDICES

Allocation Algorithm

```
// Sort courses by units (descending) to allocate larger courses first
const sortedCourses = [...courses].sort((a, b) => b.units - a.units);

for (const course of sortedCourses) {
  let bestLecturer: Lecturer | null = null;
  let bestScore = -1;

  // Find the best lecturer for this course
  for (const lecturer of lecturers) {
    // Check if lecturer has capacity
    if (lecturerWorkloads[lecturer.id] + course.units > lecturer.maxWorkload) {
      continue;
    }

    // Check if lecturer has matching specializations
    const matchingSpecs = course.specializationAreas?.filter(
      courseSpec => lecturer.specializations.includes(courseSpec)
    ) || [];

    if (matchingSpecs.length === 0) {
```

```
  continue;
}

// Calculate score based on expertise and preference
const expertiseScores = matchingSpecs.map(spec => lecturer.expertiseRatings[spec] || 50);
   const preferenceScores = matchingSpecs.map(spec => lecturer.preferenceRatings[spec] ||
50);

        const avgExpertise = expertiseScores.reduce((sum, score) => sum + score, 0) /
expertiseScores.length;
       const avgPreference = preferenceScores.reduce((sum, score) => sum + score, 0) /
preferenceScores.length;

const normalizedExpertise = avgExpertise / 100;
const normalizedPreference = avgPreference / 100;

const score = (
  (normalizedExpertise * settings.expertiseWeight / 100) +
  (normalizedPreference * settings.preferenceWeight / 100)
);

// Prefer lecturers with lower current workload (load balancing)
const workloadPenalty = lecturerWorkloads[lecturer.id] / lecturer.maxWorkload;
```

```
    const adjustedScore = score - (workloadPenalty * 0.2);

    if (adjustedScore > bestScore) {
      bestScore = adjustedScore;
      bestLecturer = lecturer;
    }
  }

  // Assign course to best lecturer
  if (bestLecturer) {
    assignments.push({
      lecturerId: bestLecturer.id,
      courseId: course.id,
      hours: course.requiredHours
    });

    lecturerWorkloads[bestLecturer.id] += course.units;
    allocatedCourses.add(course.id);
  }
}
```